



Leveraging federated learning for DoS attack detection in IoT networks based on ensemble feature selection and deep learning models

Tasneem Qasem Al-Ghadi^a, Selvakumar Manickam^a, I. Dewa Made Widia^b,
Eka Ratri Noor Wulandari^b, Shankar Karuppayah^{a,b,c,*}

^a Cybersecurity Research Centre (CYRES), Universiti Sains Malaysia, Gelugor, Penang 11800, Malaysia

^b Brawijaya University, Indonesia

^c BitRanger Sdn Bhd, Malaysia

ARTICLE INFO

Keywords:

Federated learning
DoS attack
Centralized learning
Intrusion detection system
Internet of Things

ABSTRACT

The Internet of Things (IoT) seamlessly integrates into daily life, enhancing decision-making and simplifying everyday tasks across various domains, including organizations, healthcare, the military, and industry. However, IoT systems face numerous security threats, making data protection against cyberattacks essential. While deploying an Intrusion Detection System (IDS) in a centralized framework can lead to data leakage, Federated Learning (FL) offers a privacy-preserving alternative by training models locally and transmitting only the updated model weights to a central server for aggregation. Detecting Denial-of-Service (DoS) attacks in IoT networks is critical for ensuring cybersecurity. This study compares the performance of centralized and federated learning (FL) approaches in detecting DoS attacks using four deep learning models: Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Convolutional Neural Network (CNN). To enhance model efficiency, we apply filter-based feature selection techniques, including Variance Threshold, Mutual Information, Chi-square, ANOVA, and L1-based methods, and employ an ensemble feature selection approach by combining them through a union operation. Additionally, a wrapper-based Recursive Feature Elimination (RFE) method is used to refine feature selection by removing redundant and irrelevant features. Experiments were conducted using the IoT Intrusion Dataset (IoTID20), and model performance was evaluated based on accuracy, precision, recall, F1-score, and ROC-AUC metrics. In the centralized learning scenario, the highest accuracy was achieved with GRU using Mutual Information (MI) at 99.91 %, followed by RNN with MI at 99.90 %. In the FL scenario, the highest accuracy was achieved with CNN using the ANOVA method at 99.73 %, followed by GRU with Chi2 at 99.61 %. These findings underscore the significant impact of feature selection on learning performance and provide valuable insights into optimizing deep learning-based DoS detection in IoT networks.

1. Introduction

The rapid advancement of technology has led to a significant increase in the number of interconnected digital devices, collectively known as the Internet of Things (IoT). The IoT device has surged from 31 billion in 2020 to a projected 75 billion by 2025 [54]. These devices, often equipped with sensors and actuators, exchange vast amounts of data autonomously or with minimal human intervention. IoT technology is rapidly being adopted across various sectors, including smart cities, healthcare, manufacturing, education, and agriculture, enhancing efficiency and automation [34].

However, despite the numerous benefits of IoT, its widespread adoption introduces serious security challenges. Many IoT devices rely on weak or outdated security mechanisms, making them vulnerable to unauthorized access. Advanced security solutions demand substantial resources, such as memory, storage, and computation. However, due to cost and resource constraints, manufacturers often opt for basic security measures instead [34].

Additionally, many IoT devices come with default, hardcoded credentials, making them easy targets for internet-wide scans and dictionary-based attacks [50]. The distributed and interconnected nature of IoT ecosystems further amplifies these vulnerabilities,

Peer review under responsibility of KeAi Communications Co., Ltd.

* Corresponding author.

E-mail addresses: tasneemalghadi@student.usm.my (T.Q. Al-Ghadi), selva@usm.my (S. Manickam), dewa_vokasi@ub.ac.id (I.D.M. Widia), ekaratri@ub.ac.id (E.R.N. Wulandari), kshankar@usm.my (S. Karuppayah).

<https://doi.org/10.1016/j.csa.2025.100098>

Received 24 November 2024; Received in revised form 2 March 2025; Accepted 25 April 2025

Available online 30 April 2025

2772-9184/© 2025 The Authors. Publishing Services by Elsevier B.V. on behalf of KeAi Communications Co., Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

enabling attackers to exploit weaknesses and cause significant damage.

Recent reports highlight a steady increase in cyberattacks targeting IoT networks, resulting in major disruptions and financial losses. According to the World Economic Forum's Global Risk Report 2023, cybercrime ranks as the eighth most significant global threat, with malware being a leading contributor. The global cost of cybercrime reached \$8 trillion in 2023 and is expected to rise to \$10.5 trillion annually by 2025. In parallel, the international cybersecurity market is anticipated to grow by 15 % annually, exceeding \$1.75 trillion between 2021 and 2025 [11].

Traditional security principles—confidentiality, integrity, and availability (CIA) - are no longer sufficient to address evolving threats. Modern security frameworks must also include audibility, trustworthiness, and accountability to tackle complex cyber risks [25]. Intrusion Detection Systems (IDS) is an effective defense mechanism, analyzing network packet metadata and monitoring traffic to identify potential threats before they impact the network [55].

Machine learning (ML) is increasingly employed to detect anomalies by identifying deviations from normal patterns, helping prevent and mitigate cyberattacks. However, traditional ML models often struggle with detecting subtle attack variations due to their reliance on manual feature engineering [1]. Deep learning (DL) models, on the other hand, offer automated feature extraction and higher detection rates, making them a more effective solution for enhancing security frameworks [25].

In a centralized architecture, data from all clients is transferred to a central server for processing, allowing quick and efficient access. However, this centralized model introduces significant security risks. Concentrating sensitive data in one location makes it easier for attackers to access or manipulate private information, such as health records, banking details, and personal data [1]. This creates serious privacy concerns and leads to high communication overhead, as the central server must process large volumes of data, increasing network congestion [35]. Additionally, a centralized system creates a Single Point of Failure (SPOF), making it vulnerable to disruptions or attacks. To address these security concerns and enhance ML performance, Google introduced the Federated Learning (FL) framework in 2016 [15].

Federated Learning (FL) is a collaborative machine learning approach that keeps data localized on participants' devices, preserving data confidentiality while facilitating the creation of robust ML models. As a decentralized framework, FL improves data privacy, reduces bandwidth consumption, conserves device power, and enhances network scalability by distributing computations across multiple locations. Additionally, FL reduces communication overhead [28] and minimizes transmission delays by calibrating model parameters locally before aggregating them [50]. In the FL framework, edge nodes collaborate by training models locally and transmitting updated model weights to a central server [8]. The server aggregates the updates from multiple local devices using the FedAvg optimization algorithm to construct a global model with averaged weights [28]. This global model is then broadcast back to the edge devices, and the iterative process continues until the model reaches optimal accuracy [50].

This paper presents a Federated Learning (FL)-based Intrusion Detection System (IDS) for IoT networks to detect cyber threats while preserving data privacy. Unlike traditional centralized approaches, our method enables local model training on edge devices, preventing direct data exposure to a central server. The key contributions of this work are as follows:

- Design of FL architecture for IoT networks, where models train locally on edge devices, and updated weights are aggregated on a central server to enhance privacy and intrusion detection.
- Implementation of four deep learning models—Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU) for both centralized and FL environments.

- Use of ensemble feature selection techniques to identify key features, enhancing the efficiency and accuracy of intrusion detection.
- Evaluation using the IoT Intrusion Dataset (IoTID20) to assess the effectiveness of the proposed framework.
- Comparison with state-of-the-art IDS mechanisms, demonstrating the advantages of our FL-based approach.

The rest of this paper is structured as follows: Section 2 provides background information and a review of related work. Section 3 details the proposed approach, while Section 4 covers its implementation. Section 5 discusses the experimental results, and Section 6 concludes the paper with insights and directions for future work.

2. Background and related work

In this section, we discuss key topics related to the Intrusion Detection System (IDS), the Internet of Things (IoT), machine learning (ML) algorithms, and relevant related work. Section 2.1 introduces the concept of IDS, while Section 2.2 explores IoT technologies and architecture. Section 2.3 covers the various threats faced by IoT systems, and Section 2.4 focuses on the classification of these threats using ML algorithms. Furthermore, Section 2.5 examines the challenges associated with intrusion detection in Federated Learning. Finally, Section 2.6 reviews relevant studies in the field.

2.1. Intrusion detection system (IDS)

An Intrusion Detection System (IDS) is a hardware or software application that continuously monitors a network for any breaches of policy or unauthorized access to the system. When such activities are detected, the IDS generates an alert to notify the system administrator, who can then take the necessary actions to address and mitigate the threats. IDS is typically categorized into two types based on its architecture: the Host Intrusion Detection System (HIDS) and the Network Intrusion Detection System (NIDS). HIDS focuses on monitoring internal log files of individual systems, while NIDS monitors the data traffic across all connected networks to identify potential intrusions [58].

Intrusion Detection Systems (IDS) primarily use two main detection methods. The first is signature-based intrusion detection, which generates an alert when a new data pattern matches known attack signatures stored in a library. However, this method fails to detect new, previously unseen attacks that do not exist in the library. The second method is anomaly-based intrusion detection, which is used to detect zero-day attacks. It identifies intrusions by detecting deviations in user behavior from established norms. Normal behavior is defined through statistical analysis of user activity [1,20].

2.2. IoT technologies and architecture

IoT technologies play a crucial role in improving the quality of services (QoS) and supporting decision-making processes. These systems can collect large amounts of data in real time, and they possess key characteristics such as automation, scalability, and robustness. IoT devices are widely used in smart environments, such as smart homes, healthcare, military applications, transportation, and agriculture, to enhance the comfort and convenience of daily life. However, the growing usage of IoT devices has also led to increased cybersecurity risks. Malicious devices can spy on other devices, inject harmful data to manipulate traffic, steal private information, or even disrupt the entire system [36,54].

As IoT systems have become more popular, concerns about privacy and security have intensified, with issues like device spoofing and cyberattacks emerging as major threats. Protecting IoT devices requires building strong security mechanisms to safeguard against intrusions. Additionally, the limited resources of IoT devices, such as bandwidth, memory, and computational capacity, make them vulnerable to exploitation by intruders seeking to access confidential data [25].

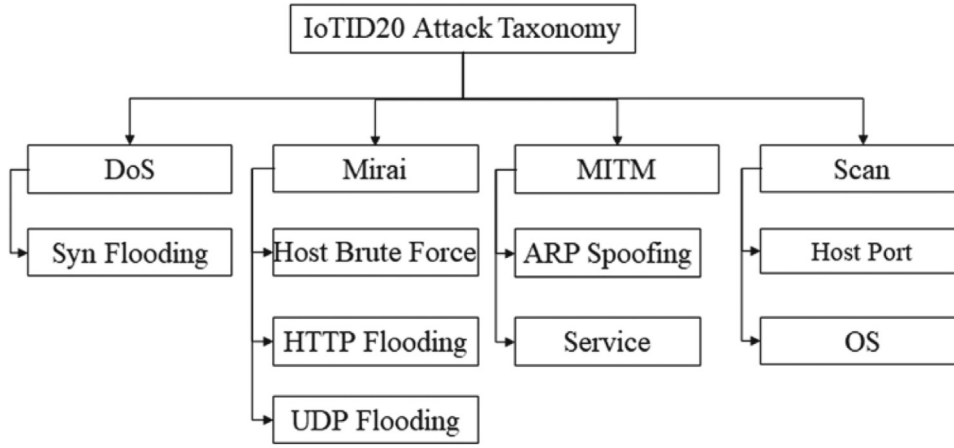


Fig. 1. Attack taxonomy of the IoTID20 dataset [60].

Table 1
Attacks on the IoT Intrusion Dataset (IoTID20).

intrusion	Intrusion types
DoS	DoS-Synflooding
Scan	Scan Port OS, Scan Hostport
Mirai	Mirai-UDP Flooding, Mirai-Hostbruteforcecg, Mirai-HTTP Flooding, Mirai-Ackflooding
MITM ARP Spoofing	MITM ARP Spoofing

An IoT platform is structured into three layers. The physical layer is made up of endpoint devices, such as sensors and actuators, which gather real-time data. The network layer facilitates communication between devices and networks, using protocols like Message Queuing Telemetry Transport (MQTT), ZigBee, Application Programming Interface (API), and Wi-Fi. The application layer, the final layer, delivers essential services to end-users, including reporting, device control, and analytics, often enhanced through machine learning (ML) to provide accurate and efficient intrusion detection [20,21].

Each layer of an IoT platform is vulnerable to specific threats. The physical layer, for instance, is exposed to jamming, phishing, Radio Frequency Identification (RFID) tracking attacks, botnets, and eavesdropping. The network layer is susceptible to attacks such as Denial of Service (DoS), Distributed Denial of Service (DDoS), replay attacks, Man-in-the-Middle (MITM) attacks, and routing attacks. On the application layer, common threats include malware, injection attacks, session hijacking, and access control issues [21,24].

2.3. Threats in IoT systems

Threats in IoT systems are typically classified into two categories based on their response: active and passive threats. Active attacks disrupt system resources and alter data streams, including threats such as Sybil attacks, device spoofing, malware, Man-in-the-Middle (MITM) attacks, and Denial of Service (DoS) attacks. On the other hand, passive threats, such as eavesdropping and phishing, involve attackers seeking to obtain sensitive data without damaging the system's resources [40]. The IoTID20 dataset includes four primary types of attacks: DoS, scan, Mirai, and MITM ARP Spoofing [60], as shown in Fig. 1. Table 1 provides an overview of the intrusion types in the IoT Intrusion dataset, IoTID20.

A Denial of Service (DoS) attack is one of the most common threats to IoT networks, where attackers flood a server with excessive traffic, preventing legitimate users from accessing network resources and denying services. This overload renders the server ineffective and limits its processing capabilities [18,20,39]. A scan attack, which is a type of probe attack, occurs when attackers scan network devices to identify

vulnerable points in the network topology, aiming to exploit them for unauthorized access [18]. The Mirai threat is a form of malware attack that infects IoT devices, transforming them into a botnet of remote-controlled bots. These bots scan for vulnerabilities in the network and launch Distributed Denial of Service (DDoS) attacks using protocols like Transmission Control Protocol (TCP), User Datagram Protocol (UDP), and Hypertext Transfer Protocol (HTTP), disrupting computer systems. Fig. 2 illustrates the infection process of the Mirai botnet. The Man-in-the-Middle (MITM) attack involves attackers establishing independent connections with victims, intercepting and manipulating messages to create a private conversation [20].

2.4. Classification with machine learning algorithms

Artificial Intelligence (AI)-based attack detection creates a dynamic security framework by utilizing Machine Learning (ML) models to identify valuable patterns in network traffic data. These models train classifiers to detect unwanted signals, with Deep Learning (DL) approaches demonstrating improved detection accuracy and a reduced False Alarm Rate (FAR). To ensure the protection of IoT networks, ML and DL models must be continuously updated to meet the evolving security needs of next-generation systems and safeguard IoT devices [36]. ML plays a crucial role in defending against network intrusions. The following subsections introduce the four prominent ML classifiers employed in our research.

2.4.1. Convolutional neural network (CNN) - Conv1D

A supervised deep learning model, such as Convolutional Neural Networks (CNN), typically consists of three layers: input, hidden, and output layers. The hidden layers incorporate convolutional, rectification, pooling, and fully connected layers. CNNs are used to identify new intrusion patterns by extracting features from data traffic, which are represented as a set of feature maps in the convolutional layer. This is achieved by applying the convolutional operation, utilizing a collection of filters (kernels), as described in Eq. (1) [49].

$$Y_{ij} = (X \cdot K)_{ij} = \sum_m \sum_n X_{(i+m)(j+n)} K_{mn} \quad (1)$$

Where X is the convolution operation, K is the kernel of size $m \times n$, Y_{ij} is the output feature map at position (i,j) .

The following layer is the rectification layer (ReLU), as described in Eq. (2). The ReLU function outputs 0 for negative inputs and returns the same value for positive inputs [56,59,61].

$$ReLU(x) = \max(0, x) \in [0, \infty] \quad (2)$$

The feature maps are then subsampled using the pooling layer, which reduces their dimensionality, minimizes overfitting, and speeds up the

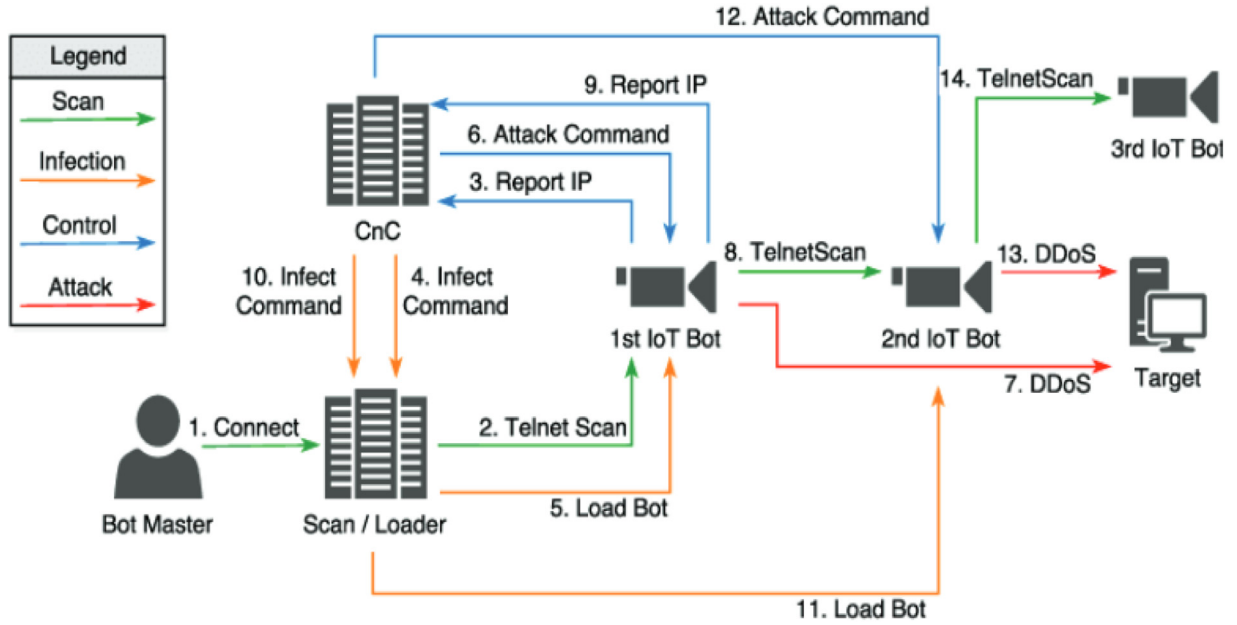


Fig. 2. Mirai Botnet infection process [46].

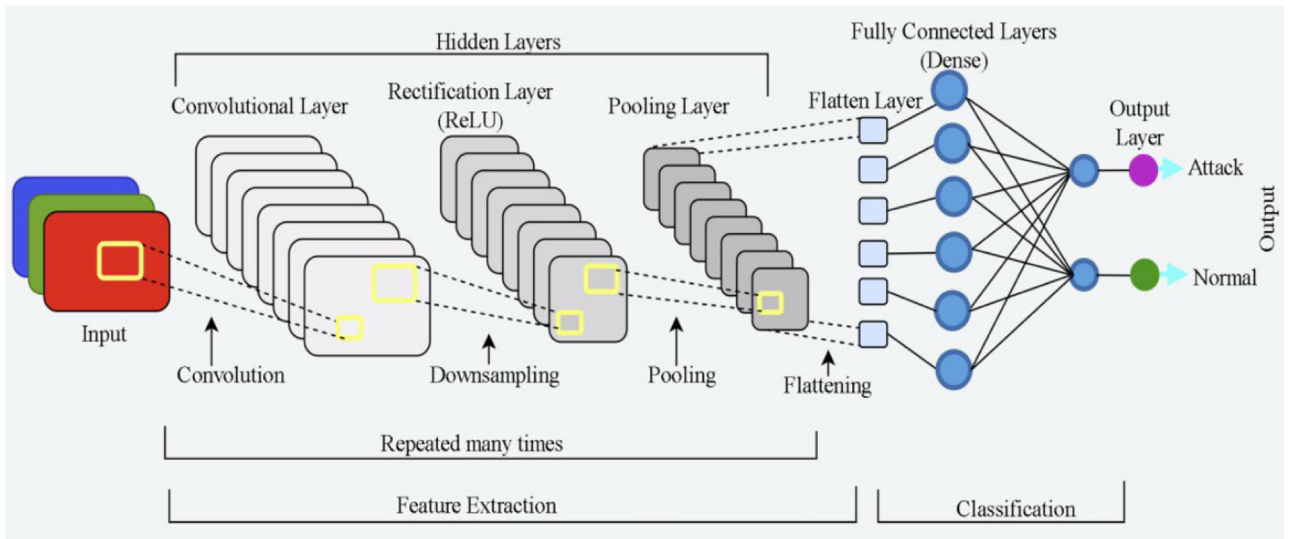


Fig. 3. Architecture of CNN for binary classification [19].

training process while retaining relevant features and eliminating outliers or inconsistencies. Following this, the identified features are arranged in a flatten layer and passed into a fully connected (dense) layer, where the sigmoid activation function is applied, as defined in Eq. (3). This activation function computes the predicted class probabilities, as shown in Fig. 3 [56,61].

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in (0, 1) \quad (3)$$

The backpropagation process in CNNs adjusts the weights to minimize the error between the predicted and actual output by calculating the gradient of the loss function [26]. CNNs are extensively used in fields like computer vision and image processing [19].

2.4.2. Recurrent neural network (RNN)

One type of supervised deep learning model is the Recurrent Neural Network (RNN). Fig. 4 illustrates the basic architecture of an RNN, where X_t represents the input vector at time step t and y_t represents

the output. The network comprises several layers that process information, with one or more recurrent loops that cycle the information over sequences [27]. This architecture is designed to capture temporal correlations in data, such as security attacks, and retain the information for temporary use. RNNs are widely employed in applications such as speech recognition, translation, semantic comprehension, computer vision, natural language processing (NLP), and text data processing [39].

The mathematical formulation of the RNN is given by Eq. (4). At each time step t , the RNN takes an input vector x_t and updates its hidden state h_t :

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h) \quad (4)$$

Where W_{xh} is the weight between the input vector and hidden layer, W_{hh} is the weight for recurrent connection, b_h is the bias vector, and $\tanh$ is the activation function [38].

The activation function, $\tanh$, as identified in Eq. (5), is utilized to non-linearly process the data, improving the model's ability to fit and

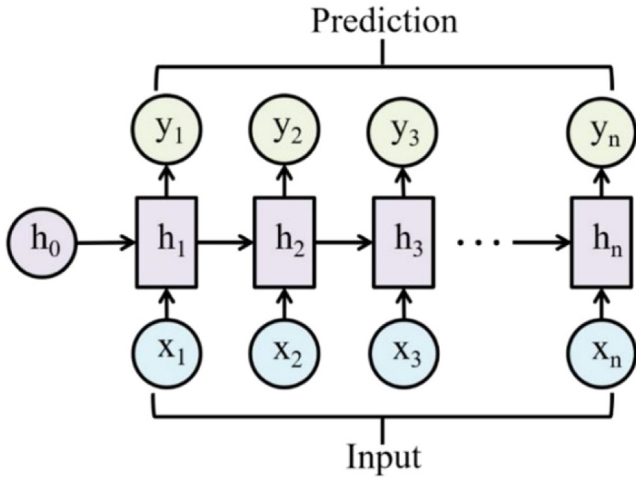


Fig. 4. Architecture of basic recurrent neural network (RNN) [38].

capture complex patterns [29,38].

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (5)$$

The output at each time step t is identified in Eq. (6):

$$y_t = \sigma(W_{hy} \cdot h_t + b_y) \quad (6)$$

Where W_{hy} is the weight between the hidden and output layers, b_y is the bias vector and σ is the activation function for the output layer [38].

Although RNNs are effective for prediction tasks, they encounter the vanishing gradient problem when processing long sequences. To address this issue, Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) were developed.

2.4.3. Long-short-term memory (LSTM)

A specific type of RNN addresses the limitations of traditional RNNs and overcomes issues like vanishing and exploding gradients. This architecture includes three gates, as depicted in Fig. 5. The input gate controls the flow of new data into the cell state, the forget gate determines which information should be discarded from the cell state, and the output gate transfers relevant portions of the cell state to the next layer. The cell state, which carries output information from previous training, regulates the flow of information and enables the network to learn and retain long-term dependencies by deciding what information to retain or discard [25,41].

The LSTM consists of three gates, each governed by the Eqs. (7)–(12) [41]. The Input gate functions according to Eqs. (7) and (8):

$$i_t = \sigma(W_{xi} \cdot x_t + W_{hi} \cdot h_{t-1} + b_i) \quad (7)$$

$$\tilde{C}_t = \tanh(W_{xc} \cdot x_t + W_{hc} \cdot h_{t-1} + b_c) \quad (8)$$

The Forget gate operates according to Eq. (9):

$$f_t = \sigma(W_{xf} \cdot x_t + W_{hf} \cdot h_{t-1} + b_f) \quad (9)$$

The Output gate operates according to Eqs. (10) and (11):

$$o_t = \sigma(W_{xo} \cdot x_t + W_{ho} \cdot h_{t-1} + b_o) \quad (10)$$

$$h_t = o_t * \tanh(C_t) \quad (11)$$

The cell state governs the memory components of the LSTM, as described in Eq. (12).

$$C_t = C_{t-1} * f_t + i_t * \tilde{C}_t \quad (12)$$

2.4.4. Gated recurrent unit (GRU)

GRU is an advanced technique that enhances RNN by optimizing the processing of sequential data. It determines how much historical data should be passed on to the next layer while retaining the relevant information over time. As shown in Fig. 6, GRU consists of two gates: the reset gate, which decides how much of the past information should be discarded, and the update gate, which controls how much of the important past data is retained and combined with new data. This structure enables GRU to handle long-term dependencies more effectively, addressing the vanishing gradient problem seen in RNNs. GRU is widely used in applications that manage information flow across time steps, such as language modeling and time series prediction [41].

The GRU consists of two gates, each governed by the Eqs. (13)–(16) [41]. The Reset gate is governed by Eq. (13):

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (13)$$

The Update gate is defined by Eq. (14):

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (14)$$

The candidate's hidden state improves the model's accuracy, as shown in Eq. (15):

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t] + b) \quad (15)$$

The new state combines the candidate's hidden state with the previous state, as depicted in Eq. (16).

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \quad (16)$$

2.5. Challenges in intrusion detection using federated learning

The goal of Federated Learning (FL) is to achieve performance equal to or better than the centralized approach while preserving privacy and ensuring efficient communication [12]. However, FL encounters many challenges, such as communication overhead, complex aggregation processes, non-IID data distribution effects, and susceptibility to adversarial attacks. The following subsections provide a detailed overview of these challenges.

2.5.1. Communication overhead

The diverse nature of IoT devices contributes to increased communication overhead, as traffic accumulates on the server while waiting for slower devices to transmit updated weights before aggregation. This delay leads to higher latency and exposes the server to potential security threats. Asynchronous Federated Learning (AFL) approaches have been introduced, allowing the server to update model weights asynchronously and accelerate convergence through novel classification mechanisms [63]. Additionally, the frequent exchange of model parameters demands high bandwidth, prolongs training time, increases energy consumption in resource-constrained environments, and slows convergence rates [1,28]. Hence, improving network communication speed and enhancing FL efficiency is crucial. This can be achieved by developing optimized communication protocols that minimize the number of communication rounds and reduce the size of transmitted data through local update parameter compression [42]. Meta-heuristic optimization algorithms have effectively reduced communication costs, enhanced latency, ensured reliable edge computation, and improved model accuracy [13].

2.5.2. Complexity of model aggregation

The server experiences a heavy traffic load, creating a bottleneck when aggregating local model parameters from numerous devices, including millions of smartphones, at the same time. While the FedAvg algorithm is commonly used for aggregating updated local model parameters, selecting the appropriate aggregation function remains challenging due to its impact on model convergence and training speed [1]. Several aggregation methods have been developed to address this issue

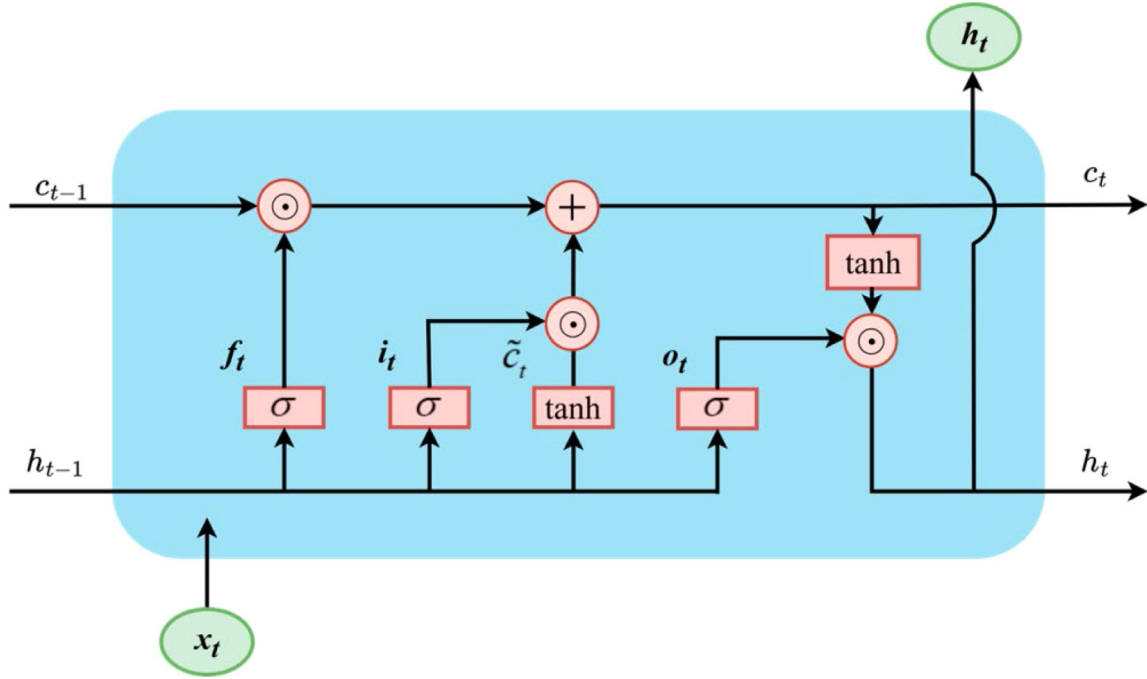


Fig. 5. Architecture of Long Short-Term Memory (LSTM) [64].

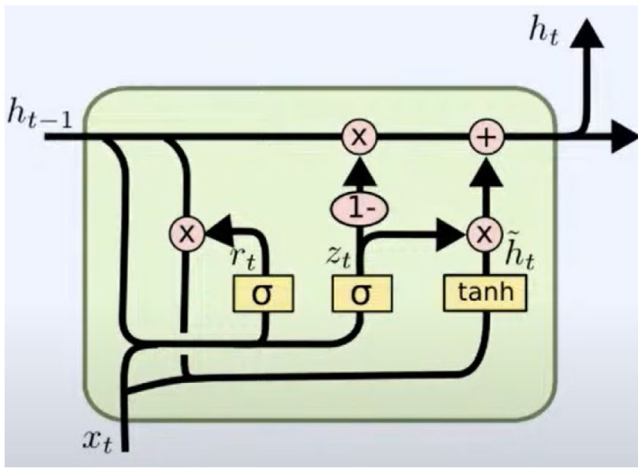


Fig. 6. Architecture of Gated Recurrent Unit (GRU) [41].

in Federated Learning (FL) environments and enhance overall performance. These aggregation algorithms include:

- FedAvg: This is the simplest approach, averaging the weights of updated local models from clients during each communication round (t) [12].
- FedProx: Short for Federated Proximal, this algorithm was introduced to address the limitations of FedAvg, particularly in heterogeneity or non-IID scenarios. It adds a proximal regularization term (μ) at the server side during the initial round to manage the weights and reduce the deviation of locally updated models from the previous global model [5,12]. This approach reduces computational overhead and improves model coherence [57].
- SCAFFOLD: Stochastic Controlled Averaging algorithm (SCAFFOLD) helps to reduce variance in both client (ck) and server (c) updates. It controls client updates by specifying the weight drift, depending on the difference between the client and server weights (ck - c) [12].

- FedSGD: Federated Stochastic Gradient Descent (FedSGD) trains local models on clients using just one batch and one epoch. However, this method requires more training time and communication, leading to slower convergence and often yielding poor performance results [57].
- FedPer: Federated Personalization is divided into two layers: a base layer on the server side and a personalized layer on the client side. This algorithm aims to mitigate issues of statistical heterogeneity in FL environments [57].

2.5.3. Managing non-independent and identically distributed (non-IID) data

The variance in datasets - stemming from differences in type, size, and complexity - can lead to asymmetric local model training results due to the heterogeneity of data sources, often resulting in high false alarms. This data heterogeneity is influenced by factors such as device types, user demographics, and environmental conditions [28]. Aggregating these diverse local models can cause underfitting in the global model [62]. Hence, various approaches have been proposed to reduce the inconsistencies arising from training machine learning models on heterogeneous data. One such solution is replacing the FedAvg algorithm with FedProx, which introduces a regularization term to each local objective function, helping to achieve greater stability [5,9] proposed clustering clients into groups based on similar updated weights. Each cluster trains independently and in parallel, with a Cluster Head (CH) selected for each group to aggregate the weights within the cluster. The CH then uploads the aggregated results to the next layer in a hierarchical manner to form a global joint model. This approach reduces server overhead, as the CH handles aggregation locally, minimizing communication rounds and costs. Additionally, it improves communication efficiency and effectively addresses the non-IID data issue.

2.5.4. Privacy challenges

FL models are vulnerable to attacks from malicious clients who tamper with model weights or data, leading to a degradation in the learning process, such as poisoning attacks. These attacks are typically categorized into data poisoning and model poisoning. In data poisoning,

attackers compromise and modify the data, often through label-flipping attacks (LF). In model poisoning, adversaries manipulate the local model gradients, altering the learning process [66]. Differential Privacy (DP) is an effective technique for protecting data and model information in FL by adding noise to the data or model parameters, typically using the Gaussian or Laplace mechanisms. This noise is introduced before training the model or sending updates to the FL server for aggregation. While DP ensures strong data privacy protection, it can result in reduced data quality and slower model convergence [43]. Other defense techniques employ robust aggregation algorithms to identify and discard malicious clients, noisy updates, and outliers during the aggregation process. These methods determine corrupted weights and address heterogeneity within the FL scenario. Examples of such techniques include trimmed mean and Byzantine-robust aggregation [4].

2.6. Related work

Several studies have explored network intrusion detection and federated learning. This section is structured into five subsections. Section 2.6.1 discusses advancements in federated learning for wireless and edge networks, highlighting its applications beyond intrusion detection. Section 2.6.2 examines machine learning-based approaches for network intrusion detection, focusing on their effectiveness in identifying cyber threats. Section 2.6.3 provides an overview of intrusion detection systems leveraging federated learning to enhance security while preserving data privacy. Section 2.6.4 addresses the challenge of communication overhead in federated learning and explores optimization techniques to improve efficiency. Finally, Section 2.6.5 emphasizes the role of Blockchain technology in ensuring the integrity and security of IoT data. A summary of recent studies is presented in Table 2.

2.6.1. Advancements in federated learning for wireless and edge networks

[23] introduced a novel federated learning (FL) receiver design aimed at enhancing symbol detection in downlink fading channels within wireless networks. The methodology involves developing two new architectures: one utilizing average pooling with a compensation layer (APCS) and the other employing a combination of average and max pooling with dense layers (APD). Federated training is utilized, where user equipment (UEs) update local models and send them to a base station for aggregation. These architectures aim to refine received signal processing by reducing data ripples and leveraging diverse pooling strategies. Simulation results demonstrate that the APD structure significantly lowers the bit error rate (BER) compared to traditional FL receivers, achieving up to a 50 % improvement for 10 UEs and 28 % for 50 UEs in an independent and identically distributed (i.i.d.) channel scenario. Additionally, the APD structure excels in non-i.i.d. environments, albeit with increased computational complexity. Similarly, [53] introduced a partial Federated Learning model for cognitive unmanned aerial vehicle (UAV)-enabled edge computing networks to minimize latency in IoT communications. The proposed framework leverages a cognitive UAV (CUAV) as a mobile relay, enabling efficient data exchange between IoT devices and an edge computing server. By distributing computation between the CUAV and the base station, the partial FL model reduces both data transmission and centralized processing. Simulation results indicate that this approach lowers latency by 21.72 % compared to the traditional FL model, with greater efficiency gains as bandwidth increases, making it a viable solution for enhancing edge computing in UAV-assisted IoT networks.

In addition, [10] introduce Federated-WDCGAN, a novel data generation framework that integrates federated learning with an enhanced generative adversarial network (GAN) to produce realistic energy consumption data while safeguarding privacy. This decentralized approach ensures data remains with its owners, minimizing privacy risks. The framework features a centralized generator for data synthesis and multiple distributed discriminators for authenticity validation, incorporating Wasserstein distance to improve GAN training stability. Experimental

results indicate that the generated data retains essential statistical properties of real data and performs well in household characteristic classification, establishing it as a robust solution for privacy-preserving data sharing in smart energy management.

2.6.2. Machine learning-based network intrusion detection

Several intrusion detection algorithms based on traditional machine learning models have been proposed. For instance, the research by [51] focused on addressing data dimensionality by applying filter and wrapper feature selection techniques. They introduced deep learning models to detect attacks in IoT networks, incorporating ensemble feature methods to enhance model performance. They evaluated four models - CNN, RNN, LSTM, and GRU using the IoT Intrusion IoTID20 dataset. Their approach achieved detection accuracy between 97.05 % and 97.87 %, precision ranging from 96.99 % to 97.95 %, recall from 99.80 % to 99.95 %, and F1-measure between 99.45 % and 99.87 % across various models. Furthermore, [52] proposed a CNN-based Intrusion Detection System (IDS) to detect abnormal traffic on the NID and Bot-IoT datasets, achieving accuracies of 99.51 % and 92.85 %, respectively. However, [55] applied Recursive Feature Elimination (RFE) to three machine learning algorithms - Decision Tree (DT), Support Vector Machine (SVM), and Random Forest (RF) - using the KDD CUP99 dataset. They concluded that SVM outperformed both RF and DT algorithms with an F1-measure of 100 % for U2R attacks, compared to 91 % and 88 % for RF and DT, respectively, for the same attack type. Similarly, [18] recommended using XGBoost with RFE for the NSL-KDD dataset. Their approach achieved F1 scores of 99.72 %, 99.53 %, 97.67 %, and 93.21 % for DoS, Probe, U2L, and U2R attack types, respectively [3] used the Mutual Information (MI) feature selection method on Deep Neural Networks (DNN) to detect attacks in the IoT-Botnet 2020 dataset, achieving superior performance with an accuracy of 99.01 %, compared to other deep learning models like RNN, CNN-1D, GRU, and LSTM, which had an accuracy of 98.43 %, 98.88 %, 98.39 %, and 96.41 %, respectively. Likewise, [39] proposed DNN and CNN with RFE for classification using the NSL-KDD dataset. Their approach achieved 94 % accuracy for binary classification with DNN, while RNN was used for multi-class classification, providing accuracy of 92 %, 96 %, 87 %, 69 %, and 94 % for normal instances, DoS, Probe, R2L, and U2R, respectively.

However, [32] introduce a novel Network Intrusion Detection System (NIDS) tailored for cloud environments, utilizing the Transformer model's attention mechanism to enhance detection accuracy. Their approach involves preprocessing the CIC-IDS 2018 dataset and training a Transformer encoder-based model incorporating multi-head attention layers, feedforward layers, and layer normalization. To optimize performance, they progressively increase the number of encoder layers. Experimental results indicate that the proposed model achieves over 93 % accuracy, rivaling existing CNN-LSTM models, and effectively detects various intrusion types, including DoS, DDoS, and Botnets while reducing false positives in cloud security. This study highlights the potential of Transformer-based models in strengthening network defense mechanisms.

2.6.3. Federated learning-based intrusion detection

The study by [50] employed the Federated Learning (FL) architecture to ensure user data privacy while detecting attacks on the Edge-IoTset dataset. Both centralized and FL approaches were implemented using CNN and RNN models. The performance evaluation, which assessed benign and attack instances separately, showed accuracy values of approximately 93 % and 94 %, precision values of 94 % and 95 %, recall values of 93 % and 95 %, and F1-scores of 93 % and 94 % for CNN and RNN, respectively.

In the FL scenario, the experiments were repeated with varying numbers of clients - three, nine, and fifteen - across two case studies: Independent and Identically Distributed (IID) and Non-Independent and Identically Distributed (Non-IID). As the number of rounds increased,

Table 2
Overview of State-of-the-Art techniques in federated learning.

Ref.	Approach	Key Achievements	Privacy-Preserving	Reduced Overhead	Handles Heterogeneous Data	Scalability Focus
[23]	Federated learning receiver with APCS and APD architectures.	Improved symbol detection; APD reduces BER by up to 50 % (10 UEs) and 28 % (50 UEs).	×	✓	✓	×
[53]	Partial Federated Learning with UAV-enabled edge computing.	Reduced latency by 21.72 % compared to traditional FL.	✓	✓	✓	✓
[10]	Federated-WDCGAN (Federated Learning and Improved GAN).	Generates realistic energy consumption data while preserving privacy; retains key statistical properties; improves classification of household characteristics.	✓	×	✓	×
[51]	Feature selection (filter & wrapper) + DL models (CNN, RNN, LSTM, GRU).	Improved detection accuracy (97.05 %–97.87 %) via ensemble feature methods.	×	✓	✓	×
[52]	CNN-based IDS.	Achieved high accuracy (99.51 %, 92.85 %) for intrusion detection.	×	✓	×	×
[55]	Recursive Feature Elimination (RFE) with DT, SVM, RF.	SVM outperformed RF and DT with 100 % F1-measure for U2R attacks.	×	✓	✓	×
[18]	XGBoost with RFE	Achieved high F1 scores (99.72 %–93.21 %) for different attack types.	×	✓	✓	×
[3]	Mutual Information (MI) feature selection and DNN	DNN outperformed other DL models, achieving 99.01 % accuracy.	×	✓	✓	×
[39]	DNN and CNN with RFE.	Achieved 94 % accuracy (binary classification) and varied multi-class results.	×	✓	✓	×
[32]	Transformer-based NIDS for Cloud.	Achieves 93 % accuracy in intrusion detection; outperforms CNN-LSTM models; reduces false positives; detects DoS, DDoS, and Botnets.	×	✓	✓	×
[50]	FL with CNN and RNN.	FL achieved accuracy comparable to centralized models (>90 %) while preserving privacy.	✓	×	✓	✓
[61]	FL-based DNN with Mutual Information (MI) feature selection.	FMI-DNN outperformed conventional FL models (99.34 % accuracy).	✓	×	✓	✓
[56]	FL-based IDS with CNN.	FL achieved accuracy close to centralized (~81 % & ~78 %) while ensuring privacy	✓	×	✓	✓
[17]	2DF-IDS (Decentralized FL and Differential Privacy).	Enhanced privacy, accuracy (94.37 % centralized, 93.91 % FL), and improved F1 score.	✓	✓	✓	×
[29]	DAFL (Dynamic Weighted Aggregation FL) using CNN.	Reduces communication overhead by filtering model parameters.	✓	✓	×	×
[35]	Hierarchical FL with ANN.	Improves FL efficiency & scalability, reduces server load.	✓	✓	×	✓
[47]	FL-based anomaly detection with neural networks.	Aggregates multiple models and applies greedy feature selection.	×	✓	✓	×
[22]	Fed-ANIDS (FL and Anomaly Detection) with Autoencoders	Uses FedProx for better privacy and reduced false alarms.	✓	✓	✓	×
[37]	Federated Learning and Blockchain Security.	Enhances FL security by using blockchain for tamper-resistant data storage; employs SHA-256, RSA encryption, and PBFT consensus for transaction validation.	✓	×	✓	×
[7]	Blockchain Edge of Things (BEoT) for resource-restricted IoT devices.	Increased security, processing speed, resource availability, and reduced overhead.	✓	✓	✓	✓
[31]	ML and Blockchain with Differential Privacy.	Enhances IoT data security by integrating differential privacy and blockchain for confidentiality and integrity.	✓	×	✓	×

the model's performance improved, with FL achieving accuracy comparable to the centralized approach while preserving data privacy. In the 50th round, the accuracy surpassed 90.73 % and 91.87 % for CNN and RNN, respectively, when using three and nine clients.

Similarly, [61] proposed an FL-based deep neural network (DNN) model utilizing the mutual information (MI) feature selection method for effective anomaly detection on the IoT-Botnet 2020 dataset. They compared the results with conventional FL models such as LSTM, DNN, and AE. The proposed FMI-DNN achieved an impressive accuracy of 99.34 %, while the accuracy of other FL models - FLSTM, FDNN, and FAE - was 93.5 %, 95.7 %, and 97.8 %, respectively. In comparison, the centralized model achieved an accuracy of 97.28 %.

Similarly, experiments conducted on the UNSW-NB15 dataset [56] demonstrated the effectiveness of an FL-based IDS framework that ensures data privacy. Using the CNN model, the FL system achieved accuracy close to that of the centralized system, reaching 81.19 % after 20 rounds. The FL architecture involved 15 clients, each with 10,000 randomly selected data instances. In the case of the CICIDS2018 dataset, the centralized system achieved an accuracy of 98.77 %, while the FL

system, using 10 clients and 10,000 randomly selected data instances per client, achieved an accuracy of 78.46 %.

A study by [17] addressed privacy concerns in industrial smart IoT systems by proposing a secure, Decentralized, Differentially Private (DF) Federated Learning (FL)-based Intrusion Detection System (IDS) called 2DF-IDS. This approach consists of three main components: a key exchange protocol, a differential private weights exchange technique, and a decentralized FL framework. When applied to an IoT/IIoT dataset, 2DF-IDS achieved an accuracy of 94.37 % in a centralized setting and 93.91 % in FL. Furthermore, 2DF-IDS improved the F1-score, recall, and precision by 12 %, 13 %, and 9 %, respectively, compared to other FL-based IDS frameworks.

2.6.4. Communication overhead in federated learning

One of the key challenges in Federated Learning (FL) is the growing computational demands on edge devices. To address this, [29] introduced the Dynamic Weighted Aggregation Federated Learning (DAFL) scheme, which detects intrusions in networks while minimizing communication overhead by dynamically filtering local model parameters.

DAFL, based on Convolutional Neural Networks (CNN), suspends communication with the central server and avoids uploading local model weights if the accuracy of a client falls below a detection accuracy threshold. This approach achieved impressive results, with accuracy, precision, recall, and F1-score reaching 93.24 %, 91.66 %, 94.20 %, and 92.90 %, respectively. In contrast, [35] proposed a hierarchical FL framework based on Artificial Neural Networks (ANN) to detect network anomalies using the UNSW-NB15 dataset. This approach improves the scalability and efficiency of FL while reducing the load on the central server. The ANN-HFL framework varied the number of clients (4, 6, and 10) and edge servers (4, 3, and 3) and achieved accuracy rates of 96.02 %, 96.19 %, and 96.39 %, respectively, compared to 92.01 % in centralized ANN with the same dataset. Similarly, [47] suggested an anomaly-based intrusion detection framework using FL, which aggregates multiple global models and applies neural networks to evaluate classification performance over the NSL-KDD dataset, containing four types of attacks: DoS, Probe, R2L, and U2R. The dataset was distributed across eight devices, with one attack type placed on two devices integrated with normal data for training. The results showed superior performance under the greedy feature selection method in FL, achieving 70.4 % accuracy, compared to 45.8 % and 50.5 % for all 41 features using single and multiple global models, respectively. The system achieved 89.1 %, 94.5 %, 84.8 %, and 99.5 % accuracy for DoS, Probe, R2L, and U2R using greedy feature selection, while accuracy dropped to 63.4 %, 68.9 %, 78.3 %, and 97.3 % with all 41 features.

Moreover, [22] proposed the Fed-ANIDS approach, integrating FL and Anomaly Detection (AD) to detect intrusions and address inconsistencies caused by training machine learning models on heterogeneous data, FedProx was introduced as an alternative to the FedAvg algorithm. FedProx stabilizes the system by adding a regularization term to each local objective function. This approach calculates intrusion scores using three autoencoder models - simple autoencoder (AE), variational autoencoder (VAE), and adversarial autoencoder (AAE). Fed-ANIDS utilized the FedProx algorithm for aggregating the updated weights, resulting in superior performance compared to the FedAvg algorithm. It provided enhanced privacy, reduced false alarms, and better privacy preservation. Additionally, Fed-ANIDS outperformed other GAN-based models significantly.

2.6.5. Ensuring integrity of IoT data through blockchain

Given the diversity of IoT devices, various solutions have been developed to ensure the security and transparency of communication between multiple nodes. Blockchain technology has emerged as a key solution, enabling reliable peer-to-peer connections without the need for a trusted intermediary [37]. In the Federated Learning (FL) architecture, clients leverage blockchain to establish a secure platform by storing raw data in transactions that are difficult to hack or tamper with. Blockchain technology facilitates this by creating new transactions where the sender generates a transaction hash using the SHA-256 function and encrypts the transaction with the RSA algorithm. On the receiver side, there are two verification mechanisms: first, the transaction is decrypted to verify the signature and add the transaction to the block; second, the Practical Byzantine Fault Tolerance (PBFT) consensus algorithm is applied to validate the new block before adding it to the block list and broadcasting it across the network [37]. Additionally, [7] utilized Blockchain Edge of Things (BEoT) to address the limitations of IoT devices, such as low bandwidth and limited storage capacity. BEoT enhances security, accelerates processing speed, ensures resource availability by managing transactions at the edge, and reduces computational overhead during data processing.

Furthermore, [31] introduced an innovative approach to safeguarding the privacy and integrity of multimodal data generated by Internet of Things (IoT) systems through the integration of machine learning (ML) and blockchain technology. The proposed system leverages differential privacy techniques to ensure data confidentiality while utilizing blockchain to maintain data integrity. By combining these tech-

nologies, the framework empowers data owners with customizable protection mechanisms, fostering secure data sharing and enhancing user trust. This research represents a significant step forward in addressing IoT data security challenges by seamlessly unifying privacy and integrity protections within a decentralized architecture.

3. Proposed approach

In this research, we propose a Federated Learning (FL)-based Intrusion Detection System (IDS) to detect Denial of Service (DoS) attacks using ensemble feature selection. Fig. 7 illustrates the centralized model proposed by [51], which utilizes conventional deep learning (DL) models along with filter and wrapper feature selection methods. This approach follows a three-stage process: data pre-processing, feature selection, and DL-based IoT attack detection. Our methodology builds upon Rihan et al.'s approach, but while they implemented conventional DL on a centralized platform, we apply FL on a decentralized system. We replicate their procedures at each node in the FL platform and conduct experiments with both three and five clients. We also study how the number of clients influences the performance of the FL-based system.

3.1. Data preprocessing

This step prepares the data to enhance the quality of machine learning results by converting the dataset into a suitable format for deep learning models. It is divided into three subcategories. The first subcategory, data cleaning, removes missing and duplicate entries [6], as well as features related to patterns or network flow such as Flow_ID, Src_IP, Src_Port, Dst_IP, Dst_Port, Protocol, and Timestamp to ensure reliable, accurate, and unbiased results during the classification process [51]. The second subcategory involves transforming object values into appropriate formats, as deep learning models require numerical values for training and testing, enabling the algorithms to interpret and learn from encoded labels. The final subcategory is scaling the data to a specific range, ensuring that all feature values are treated equally during analysis. In this step, Min-Max scaling is applied, which is a linear transformation that maps the feature values between 0 and 1, preserving their relative relationships. Both the training and testing datasets were normalized to improve model performance, as demonstrated by Eq. (17) [6].

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (17)$$

3.2. Feature selection techniques

Feature selection is a vital technique in machine learning that enhances model performance by selecting informative features and removing irrelevant ones, thus reducing complexity, saving time, and preventing overfitting. It is typically divided into three methods: filter methods, wrapper methods, and embedded methods [18]. Feature selection is carried out by choosing the most important features based on their relevance scores [51], which helps reduce data dimensions and accelerate the training process. The rationale for experimenting with multiple feature selection techniques is to ensure that the model operates efficiently with various feature combinations and avoids bias. To improve feature selection performance, we applied an ensemble approach by performing a union operation, selecting unique features from five filter-based techniques, and examining the impact of an increasing number of features on deep learning outcomes. In our methodology, like [51], we identified the top 10 features from each filter technique—variance threshold (VT), mutual information (MI), chi-squared (chi2), ANOVA, and L1-based methods—then combined 30 significant features through a union operation. Finally, we applied a wrapper feature selection method called Recursive Feature Elimination (RFE) to select the top 10 relevant features. Detailed information on the feature selection techniques is provided in Section 4.2.

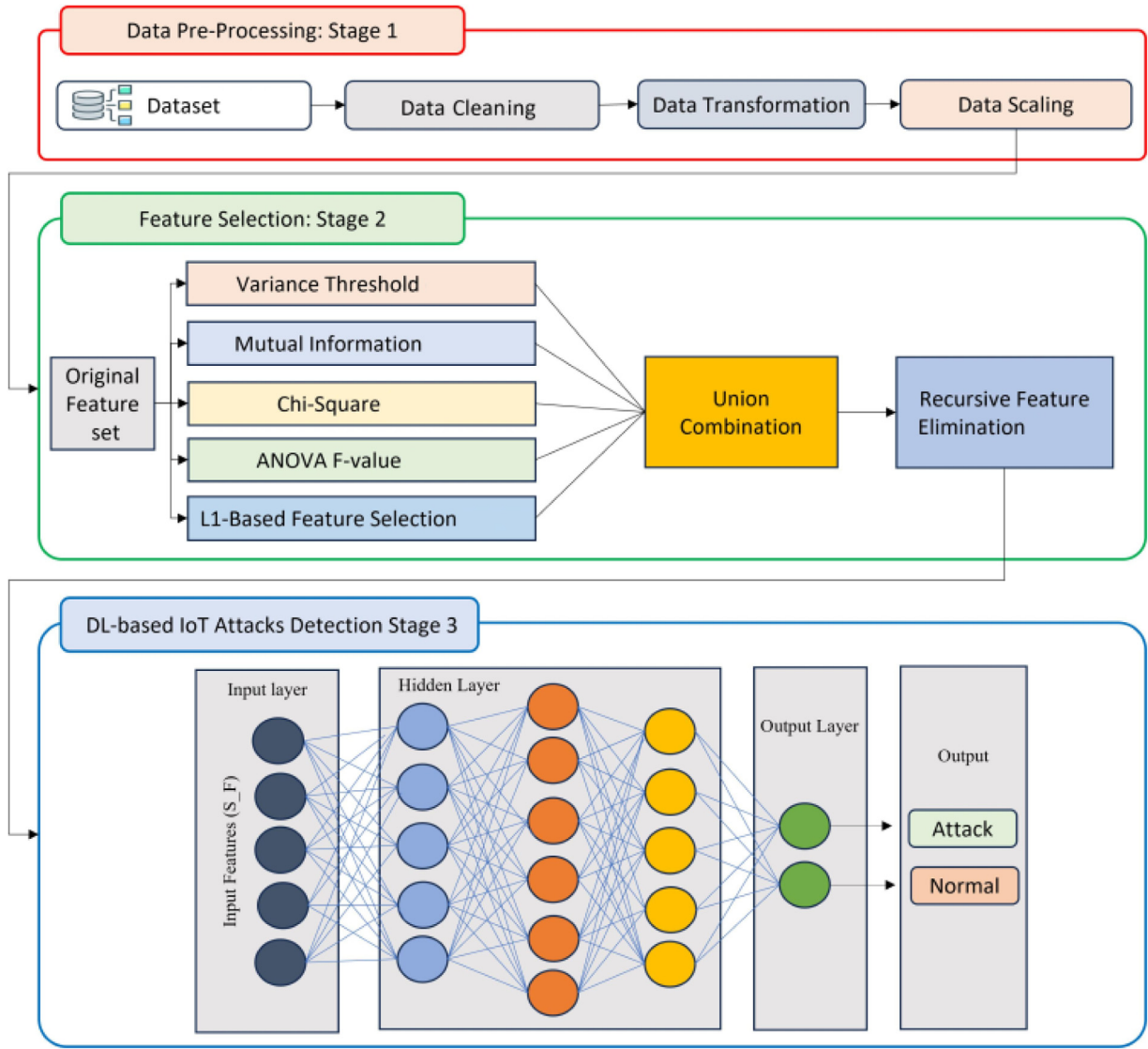


Fig. 7. Centralized model of Rihaan et al.'s approach [51].

3.3. Developing a deep learning model for intrusion detection

Four popular deep learning models, as outlined in Rihaan et al., were implemented for this research: Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), Long-Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU). The dataset was split into 80 % for training and 20 % for testing. The 80 % training set was further divided among three or five clients, depending on the experiment. Further details of the deep learning models are provided in Section 2.4.

Fig. 8 illustrates our proposed FL-based Intrusion Detection System (IDS) for IoT attacks. The model works through a series of steps, as depicted in Fig. 8, to achieve binary classification. In step 1, a global model is generated on the server side. Step 2 involves downloading the global model with initial weights to each client in the FL platform. In step 3, each client trains and refines the model using its local data. Step 4 follows with the clients uploading their locally updated models, including the updated weights, back to the server. In step 5, the server aggregates these weights using the FedAvg algorithm. Step 6 entails building the global model using the average weights and evaluating its performance by computing accuracy, precision, recall, F1-score, and AUC-ROC values. In step 7, the server broadcasts the global model to the clients, while in step 8, the model is trained with the new parameters. In step 9, the

process from steps 4 to 8 is repeated for ten rounds, after which the average values for each evaluation metric are computed across all ten rounds.

4. Implementation of the proposed system

This section outlines the implementation of the proposed approach, divided into the following subsections: Section 4.1 details the system requirements and provides the pseudocode for both centralized learning and FL models. Section 4.2 explains the feature selection techniques used. Section 4.3 presents the experimental configuration.

4.1. System requirements and setup

The details of the experimental environment and tools are provided in Table 3. The primary implementation tool used for execution and evaluation was Google Colab, with the CPU selected as the hardware.

We were unable to obtain the source code for the centralized model approach from [51]. Therefore, we reconstructed the code based on the architectural information available in their paper. (The source code for both Rihaan et al.'s approach and ours will be made publicly available after the acceptance of this paper.) The algorithm for the proposed central-

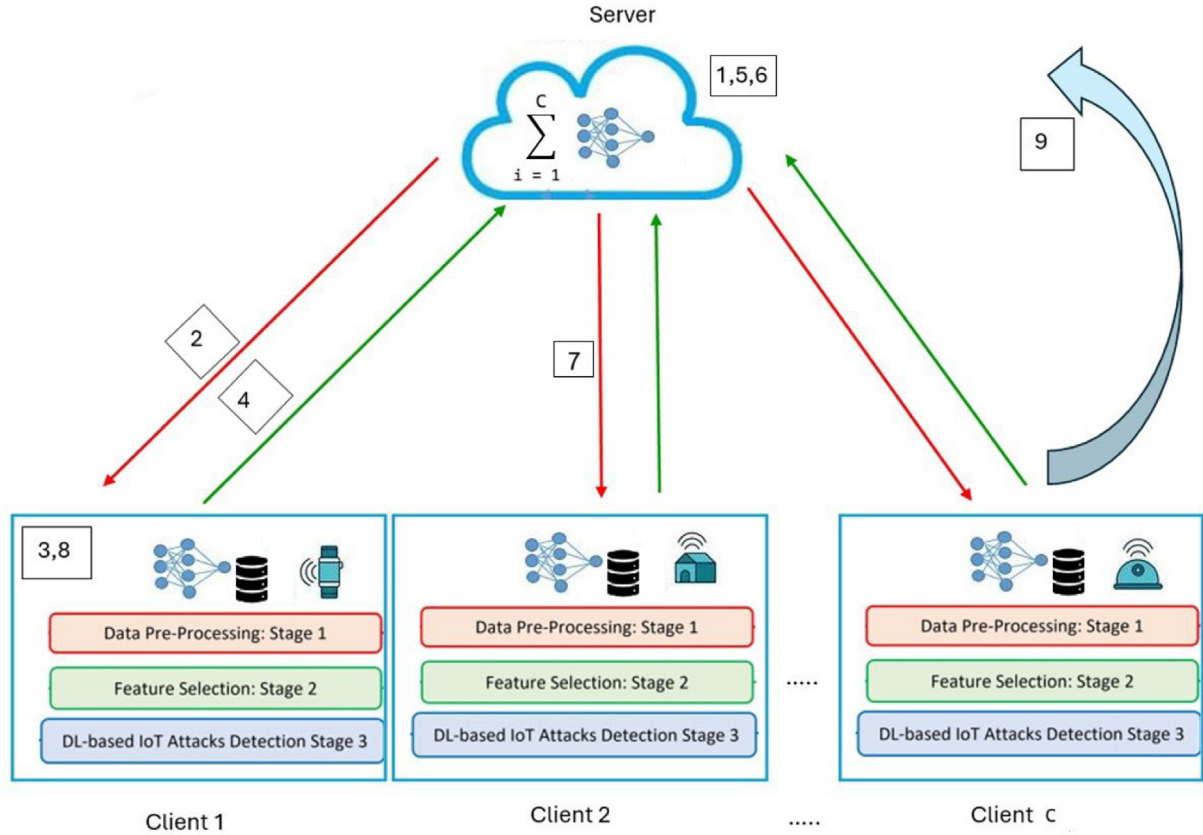


Fig. 8. Proposed FL-based approach.

Table 3
Experimental environment.

Project	Environment
Operating system	Windows 11
CPU	I5-8250U
Memory	4 GB
Python	version 3.10.12
Scikit-learn	version 1.3.2

Algorithm 1

DL-based Model for IoT Attack Detection in Centralized Learning.

Input: $X_{train}, X_{val}, X_{test}, y_{train}, y_{val}, y_{test}$
Output: Performance evaluation metrics
define DL models
Compile the DL models
Fit the DL models:
For i **in** 1 to n **do**
 While stopping criteria are not met **do**
 Train the DL models
 End while
End for
Evaluate the trained DL models
Plot the training and validation loss, confusion matrix, and ROC curve

ized approach is provided in Algorithm 1. The centralized platform defines the DL models architectures then compiled using the binary cross-entropy loss function, and trained until the early stopping criteria are met or the maximum number of local epochs n is reached. Finally, the models are evaluated. The training process is implemented separately for each feature set across different feature selection techniques.

In the Federated Learning (FL) platform, the training dataset is uniformly distributed among C clients. A directory named “central server” is created to store the trained FL models in the .keras format, while another directory, “server,” is designated for saving the model’s updated weights in the .npz format, as outlined in Algorithm 2.

Fig. 9 illustrates the flowchart of the FL system procedure. During each global epoch, the algorithm checks whether the FL models for each client ID exist in the “central server” directory and uploads them if found. It also verifies the availability of the average weights - if they are present, they are applied to the model before training. If the models are not found in the directory, it is assumed to be the first global epoch at the first round. In this case, the system defines and compiles the deep learning model’s architecture using the binary cross-entropy loss function and the Adam optimizer. Training continues either early stopping criteria are met or the maximum number of local epochs n is reached. At the end of each round, the updated weights for each client are saved in the “server” directory. The global model is then constructed by computing the average weights using the FedAvg algorithm. Finally, the models are evaluated, and the trained FL models are stored in the “central server” directory. This process is repeated for a total of ten rounds. Once all ten rounds are completed, the algorithm computes the average metrics across them.

4.2. Feature selection techniques

We applied both filter and wrapper methods to select the most relevant features for the training process and reduce the dataset’s dimensionality [51]. The filter feature selection methods include Variance Threshold (VT), Mutual Information (MI), Chi2, ANOVA, and L1-based techniques. These methods were combined using a union operation to identify the most important features across all filter methods. Finally, we

Algorithm 2

DL-based Model for IoT Attack Detection in Federated Learning.

Input: split the data between C clients
Output: Performance evaluation metrics

For rounds_num in 1 to R **do**
 For global_epochs_num in 1 to E **do**
 For client_id in 1 to C **do**
 Check if FL models (client_id) exist in the Central Server directory
 upload it
 check if avg_weights is not none
 Set the **new averaged weights** to the model before training
 else
 define DL models
 Compile the DL models
 Fit the DL models:
 For i in 1 to n **do**
 While stopping criteria are not met **do**
 Train the DL models
 End while
 End for //number of local epochs
 Save the **update weights** in the “Server” directory
 End for //number of clients
 End for //number of global epochs
 Perform federated averaging
 Build the global model with the averaged weights
 Evaluate the federated models
 Save the **trained FL models** in the “Central Server” directory
End for //number of rounds
 Calculate the averages of global metrics (Accuracy, precision, Recall, F1-score, AUC,ROC)
 Plot the training and validation loss, confusion matrix, and ROC curve

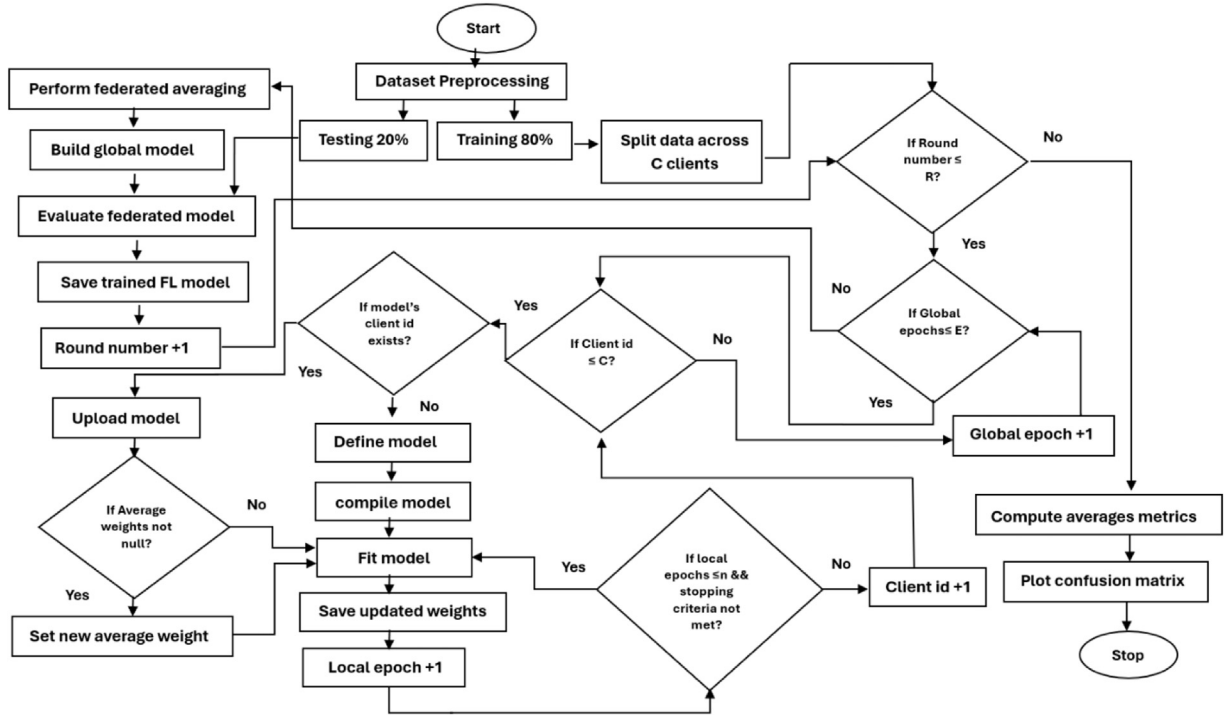


Fig. 9. Flowchart of the federated learning system.

employed the wrapper technique, Recursive Feature Elimination (RFE), to further refine the feature selection.

1. Variance Threshold (VT) is a filter feature selection technique that removes features with zero variance by default, meaning all observations of a feature have the same value across all instances. It also eliminates features whose variance falls below a specified threshold. While VT is simple, it is not highly recommended as it overlooks the correlations between features and between features and the target class. VT assumes that features with higher variance contain more useful information, which can aid the classifier in achieving optimal results [14].

The formula for VT is provided in Eq. (18) [48].

$$V_n = \frac{\sum_{i=1}^n (X_i - \mu)^2}{n} \quad (18)$$

x_i represents the individual observation, n represents the total number of observations, μ identifies the mean of the dataset.

2. Mutual Information (MI) is a statistical measure that quantifies the relationship between two random variables. MI is always non-negative, ensuring that the selected features contain meaningful information about the specific class. Mathematically, MI is expressed in

Table 4
Key features for DoS detection in the IoT intrusion dataset.

Feature selection technique	Features
VT	"Fwd_Pkt_Len_Max", "Fwd_Pkt_Len_Min", "Fwd_Pkt_Len_Mean", "Bwd_Pkt_Len_Max", "Bwd_Pkt_Len_Min", "Bwd_Pkt_Len_Mean", "Fwd_Pkts/s", "Bwd_Pkts/s", "Pkt_Len_Min", "RST_Flag_Cnt", "ECE_Flag_Cnt", "Down/Up_Ratio", "Pkt_Size_Avg", "Cat"
MI	"Flow_Duration", "TotLen_Bwd_Pkts", "Flow_Byts/s", "Flow_IAT_Mean", "Bwd_Header_Len", "RST_Flag_Cnt", "Subflow_Fwd_Byts", "Subflow_Bwd_Byts", "Active_Max", "Idle_Mean", "Cat"
Chi2	"Flow_Byts/s", "Pkt_Len_Var", "Flow_Pkts/s", "Bwd_Pkts/s", "Fwd_Pkts/s", "Bwd_IAT_Tot", "Flow_Duration", "Bwd_IAT_Max", "Flow_IAT_Max", "Idle_Max", "Cat"
ANOVA	"Bwd_Seg_Size_Avg", "Pkt_Len_Mean", "ACK_Flag_Cnt", "SYN_Flag_Cnt", "Pkt_Size_Avg", "Bwd_Pkt_Len_Max", "Bwd_Pkt_Len_Min", "Flow_Pkts/s", "Pkt_Len_Max", "Bwd_Pkt_Len_Mean", "Cat"
L1-based	"Fwd_Pkt_Len_Max", "Bwd_Pkt_Len_Max", "Bwd_Pkt_Len_Min", "Bwd_Pkt_Len_Mean", "Fwd_Pkts/s", "Bwd_Pkts/s", "Pkt_Len_Min", "RST_Flag_Cnt", "ECE_Flag_Cnt", "Pkt_Size_Avg", "Cat"
Union	"Fwd_Pkt_Len_Max", "Bwd_Pkt_Len_Min", "Idle_Mean", "Down/Up_Ratio", "ECE_Flag_Cnt", "Subflow_Bwd_Byts", "TotLen_Bwd_Pkts", "Active_Max", "Bwd_Header_Len", "Bwd_Pkts/s", "Pkt_Len_Var", "Pkt_Len_Min", "RST_Flag_Cnt", "Flow_IAT_Mean", "Fwd_Pkts/s", "Subflow_Fwd_Byts", "Flow_Byts/s", "Pkt_Len_Max", "Pkt_Size_Avg", "Fwd_Pkt_Len_Mean", "Bwd_Pkt_Len_Max", "Flow_Duration", "Bwd_Pkt_Len_Mean", "Fwd_Pkt_Len_Min", "ACK_Flag_Cnt", "Bwd_IAT_Max", "Bwd_IAT_Tot", "Bwd_Seg_Size_Avg", "Flow_IAT_Max", "Flow_Pkts/s", "Idle_Max", "SYN_Flag_Cnt", "Cat"
RFE	"Fwd_Pkt_Len_Max", "Idle_Mean", "Fwd_Pkt_Len_Std", "Bwd_Header_Len", "Bwd_Pkts/s", "Flow_IAT_Mean", "Pkt_Len_Max", "Pkt_Size_Avg", "Bwd_Pkt_Len_Max", "Flow_Duration", "Cat"

Eq. (19) [61].

$$I(R; S) = \sum_{r \in R} \sum_{s \in S} p(r, s) \log \frac{p(r, s)}{p(r)p(s)} \quad (19)$$

Where $I(R; S)$ is the MI between two random discrete variables R and S , such that $R = \{r_1, r_2, r_3, \dots, r_n\}$ and $S = \{s_1, s_2, s_3, \dots, s_n\}$ with n samples each, $p(r, s)$ represents the mass function joint probability, while $p(r)$ and $p(s)$ denotes the marginal probabilities. Based on the highest MI values, MI selects the top 10 features, which are then used as input to train the models.

3. Chi2 is a statistical test that evaluates the level of association between a feature a_i and the target class y_j by comparing the observed O_i and expected E_i frequencies. It is mathematically expressed in Eq. (20).

$$X_c^2 = \sum_{i=1}^k \left[\frac{(O_i - E_i)^2}{E_i} \right] \quad (20)$$

Where X is the Chi-square, c is the degree of freedom, O represents the observed frequencies, E represents the expected frequencies, and k represents the number of samples in the dataset [2,45].

4. ANOVA, or Analysis of Variance, is a statistical test used to assess the variance between two or more categorical groups. It is calculated as shown in Eq. (21). Features with higher variance between groups are considered more important, while those with lower variance are deemed less significant [14].

$$F = \frac{\text{between - group variance}}{\text{within - group variance}} \quad (21)$$

5. L1-based regularization, also known as Lasso (Least Absolute Shrinkage and Selection Operator) [16], adds a penalty term (L1) to the classification task. This penalty is the sum of the absolute values of the regression coefficients, multiplied by a regularization parameter λ . This process shrinks some coefficients to zero, effectively removing irrelevant features, which helps reduce overfitting and improve the model's performance [33,44,65]. The Logistic Regression cost function with regularization is provided in Eq. (22) [30].

$$j(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y_i \log(h(x_i)) + (1 - y_i) \log(1 - h(x_i)) \right] + \lambda \sum_{j=1}^n |\theta_j| \quad (22)$$

Where $j(\theta)$ is the overall loss of the logistic regression model, m represents the total number of training samples, y_i represents the actual label for i th training samples. $h(x_i)$ represents the predicted probability for i th training samples. It is computed using a logistic sigmoid function in

Eq. (23) [30]. $(1 - y_i)$ represents the complement of the actual label for i th training samples. $\log(h(x_i))$ is the logarithm predicted probability, when the exact label is 1. The $\log(1 - h(x_i))$ is the logarithm of the complement of the predicted probability when the actual label is 0. λ represents the regularization parameter (hyperparameter), the high value of λ increases the regularization and reduces the overfitting, it controls the impact of regularization the model. $\sum |\theta_j|$ is the L1-regularization term (Lasso regression) it drives some coefficient weights to zero and sparse the model.

$$h(x_i) = \frac{1}{1 + e^{-\theta^T x_i}} \quad (23)$$

6. RFE (Recursive Feature Elimination) is a wrapper-based technique that iteratively refines the model by selecting the most impactful features for intrusion detection based on their coefficients, while discarding irrelevant features through attribute ranking [39].

The main objective of feature selection is to identify the most relevant features that impact intrusion detection in IoT networks. In our implementation, we excluded identifier features used for creating network flows. Table 4 presents the key features selected through various feature selection techniques. Notably, the features chosen for VT, MI, L1-based, and RFE align with those used by [51]. However, for Chi2 and ANOVA, we introduced new features to enhance performance, leading to modifications in the selected features during the union operation.

4.3. Experimental configuration

We implemented several deep-learning algorithms and constructed the models by adjusting the hyperparameters, as detailed in Table 5. The models were compiled using the binary cross-entropy loss function, as specified in Eq. (24). Subsequently, we trained and evaluated the models on the test data, recording performance metrics and the loss function [25].

We implemented various deep-learning algorithms and optimized the models by tuning their hyperparameters, as outlined in Table 5. Each model was compiled using the binary cross-entropy loss function, as defined in Eq. (24) [25]. After training, the models were evaluated on the test data, with performance metrics and loss values recorded.

$$\text{Loss} = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})] \quad (24)$$

Where y represents the actual output and $\hat{y}$ represents the prediction output. To maintain experimental stability and consistency, we trained the data for 50 epochs in centralized learning and 5 epochs per round across 10 rounds in federated learning.

Table 5
Hyperparameter configurations for deep learning classifiers.

Classifier	parameter	value
CNN	number of hidden layers	4 (3 Conv1D + 1 dense)
	Number of dense layers	2 (one hidden, one output)
	number of hidden nodes	64, 128, 256, 128
	Filters	64, 128, 256
	dropout	0.3 - 0.1
RNN, LSTM, GRU	Kernel size	3
	dropout	0.3 - 0.2
	number of hidden nodes	128, 64, 32
	number of hidden layers	3
	Number of dense layers	1 (output layer)
Global	Batch size	2 ⁶
	Local epochs	1
	Global epochs	5
	Number of rounds	10
	Learning rate	0.001
	Loss function	Binary crossentropy
	Optimizer	Adam
	Activation function	Relu
	Classification function	Sigmoid

Table 6
Distribution of instances in the IoT intrusion dataset data.

Category	value
Total number of records	625,783
Total number of normal records	40,073
Total number of attack records	585,710
Total number of DoS records	59,391
Total number of scan records	75,265
Total number of Mirai records	415,309
Total number of MITM ARP Spoofing records	35,377
Total number of features	86
Total number of numerical features	79
Total number of categorical features	7

5. Results and discussion

This section assesses the proposed intrusion detection system for IoT networks using both centralized and Federated Learning (FL) approaches and compares their performance. The structure is as follows: [Section 5.1](#) introduces the dataset, while [Section 5.2](#) details the evaluation metrics. [Section 5.3](#) analyzes the obtained results, and [Section 5.4](#) compares the proposed approach with existing methods. [Section 5.5](#) explores the impact of increasing the number of clients on FL performance. Finally, [Section 5.6](#) summarizes the key findings.

5.1. Dataset overview

In this study, we used the publicly available IoTID20 dataset, the same dataset employed in [\[51\]](#), to facilitate a fair comparison between traditional deep learning (DL) and Federated Learning (FL) approaches. The dataset is provided in CSV format and is derived from Pcap files of the IoT botnet dataset. It includes various attack types such as DoS, Scan, Mirai, and MITM ARP Spoofing. For our analysis, we chose all benign samples from the original dataset, while the anomaly class consisted exclusively of DoS attack samples. The distribution of benign and attack records from the original dataset is shown in [Table 6](#).

For this study, the data was evenly split with 80 % allocated for training and 20 % for testing in a centralized manner. In Federated Learning (FL), 80 % of the training data was distributed across three and five clients, as shown in [Table 7](#).

5.2. Evaluation metrics

In the proposed binary classification model, we implemented standard evaluation metrics to evaluate the framework performance. These

Table 7

Data distribution for centralized and federated learning across three and five clients.

Number of clients	Training [80 %]			Testing [20 %]	
	Normal	DoS	Total	Normal	DoS
Centralized	32,196	47,375	79,571	7,877	12,016
C = 3	10,690	15,800	26,490	8,000	11,988
Total Samples	32,070	47,400	79,470	19,988	
C = 5	6,400	9,500	15,900	8,068	11,886
Total Samples	32,000	47,500	79,500	19,954	

Table 8
Confusion matrix.

Predicted class			
Actual class	Attack		Benign
	Attack	TN	FP
	Benign	FN	TP

metrics, derived from the confusion matrix, include accuracy, precision, recall, F1-score, and AUC-ROC. The confusion matrix, shown in [Table 8](#), defines the classification outcomes: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN), corresponding to the benign and attack classes [\[11\]](#).

True Positive (TP): The model correctly predicted the normal.

True Negative (TN): The model correctly predicted the attack.

False Positive (FP): The model incorrectly predicted the normal.

False Negative (FN): The model incorrectly predicted the attack.

Accuracy: is the ratio of correctly classified records to the total number of records indicated in [Eq. \(25\)](#).

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (25)$$

Precision: is the ratio of correctly predicted normal records to the total number of predicted normal records, indicated in [Eq. \(26\)](#).

$$Precision = \frac{TP}{TP + FP} \quad (26)$$

Recall: is the ratio of correctly predicted normal records to the total number of records in the normal class, as indicated in [Eq. \(27\)](#).

$$Recall = \frac{TP}{TP + FN} \quad (27)$$

F1-Score: the harmonic mean of the Precision and Recall measure. It examines the system performance using a statistical technique, indicated in [Eq. \(28\)](#).

$$F1 - score = 2 \times \frac{Precision \cdot Recall}{Precision + Recall} \quad (28)$$

The Receiver Operating Characteristics (ROC) curve is a representative graph to show the trade-off between TPR and FPR for various threshold values, the performance of the classification model is better if the AUC is higher, as indicated in [Eq. \(29\)](#) [\[39\]](#).

$$AUC = \int_0^1 \frac{TP}{TP + FN} d \frac{FP}{FP + TN} \quad (29)$$

5.3. Results analysis

In our centralized approach, we observed that Mutual Information (MI) consistently outperformed other feature selection techniques across all models. Specifically, MI achieved an accuracy of 99.91 % in GRU, 99.90 % in RNN, 99.88 % in LSTM, and 99.87 % in CNN. Chi-Square (Chi2) emerged as the second-best feature selection technique, yielding the highest accuracy in RNN and GRU at 99.88 %, followed by 99.87 % in LSTM and 99.85 % in CNN. Conversely, the lowest performance was observed with Variance Threshold (VT), achieving an accuracy of 99.48 % in LSTM, 99.50 % in GRU, and 99.52 % in both CNN and

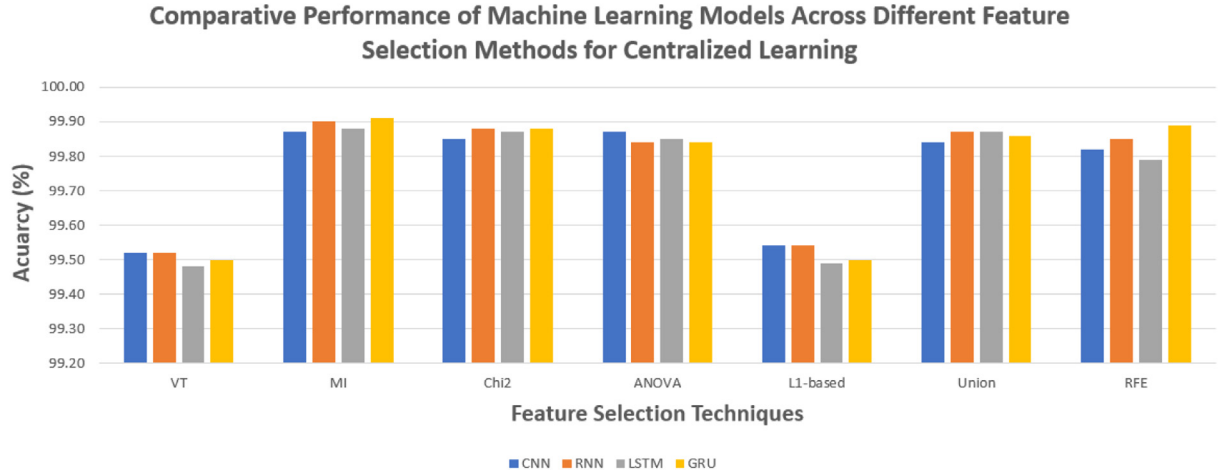


Fig. 10. Comparative performance of machine learning models across different feature selection methods for centralized learning.

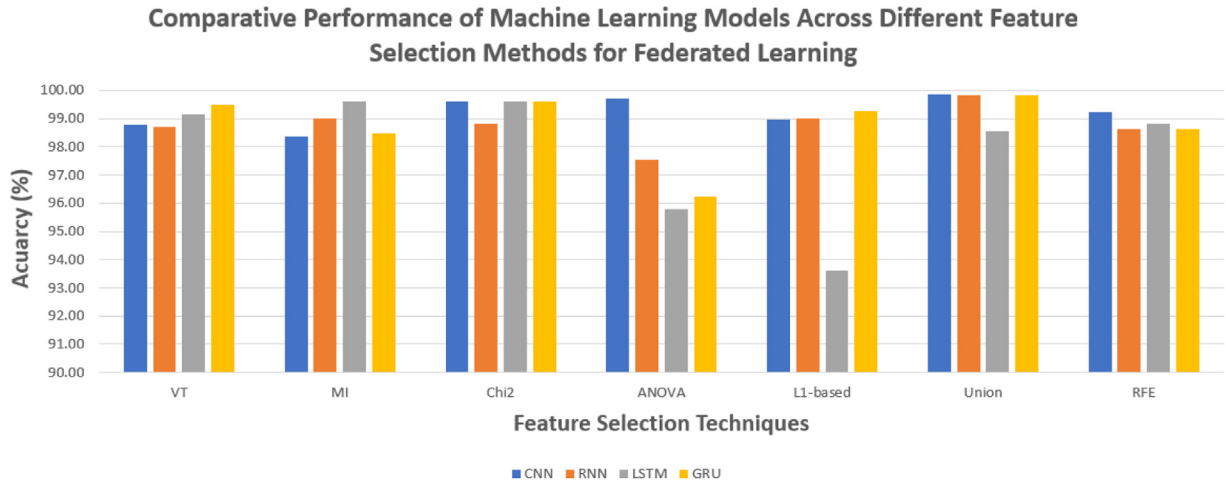


Fig. 11. Comparative performance of machine learning models across different feature selection methods for federated learning.

RNN. The second-lowest performance was recorded with the L1-based method, with accuracy values of 99.49 % in LSTM, 99.50 % in GRU, and 99.54 % in both CNN and RNN. As illustrated in Fig. 10.

Fig. 11 illustrates the FL approach, where we observed that Chi-Square (Chi2) outperformed other feature selection techniques across all models. Specifically, Chi2 achieved the highest accuracy, with 99.61 % in GRU, 99.60 % in both CNN and LSTM and 98.81 % in RNN. Variance Threshold (VT) emerged as the second-best feature selection technique, yielding the highest accuracy of 99.49 % in GRU, followed by 99.17 % in LSTM, 98.79 % in CNN, and 98.72 % in RNN.

The lowest performance was recorded with ANOVA, except for CNN, which achieved a high accuracy of 99.73 %. However, the other three models showed the lowest accuracy values, with 95.78 % in LSTM, 96.23 % in GRU, and 97.54 % in RNN. The second-lowest performance was observed with the L1-based method, with accuracy values of 93.63 % in LSTM, 98.97 % in CNN, 99.02 % in RNN, and 99.26 % in GRU.

The performance of various ML algorithms in both centralized and FL scenarios, utilizing different feature selection techniques (VT, MI, Chi2, ANOVA, L1-based, union operation, and RFE), is evaluated in terms of Accuracy, Precision, Recall, and F1-score, as presented in Table 9. The results indicate that centralized learning consistently outperforms FL across all models and feature selection techniques, albeit with a slight margin.

5.4. Comparison of our proposed approach with existing work

From Table 10, we noted that our designed centralized learning is superior to the [51] in all evaluation metrics across various DL models and feature selection techniques due to design different architecture of DL models and utilizing parameters settings vary from their approach, whereas not all parameters values identified explicitly in their research [51].

From Table 10, it is evident that our proposed centralized learning approach outperforms the method presented by [51] across all evaluation metrics, using various deep-learning models and feature selection techniques. This superiority can be attributed to the distinct architecture of our deep learning models and the different parameter settings employed, which vary from those in Rihan et al.'s approach. Additionally, not all parameter values were explicitly defined in their research.

Table 11 compares our proposed model with related FL-based work across various aspects. We specifically analyzed the accuracy values and found that our model outperforms the others.

5.5. Performance of the federated learning platform with increasing client numbers

In this section, we discuss the performance of the FL model while varying the number of clients. We compare the FL model with three

Table 9

Performance comparison of centralized and FL-based intrusion detection using different feature selection techniques (VT, MI, Chi2, ANOVA, L1-Based, Union Operation, and RFE).

Feature selection technique	model	Centralized					FL				
		Accuracy	Precision	Recall	F1_Score	AUC_ROC	Accuracy	Precision	Recall	F1_Score	AUC_ROC
VT	CNN	99.52	98.90	99.89	99.40	99.58	98.79	99.04	97.94	98.38	98.64
	RNN	99.52	98.89	99.92	99.40	99.59	98.72	99.13	97.69	98.28	98.55
	LSTM	99.48	98.88	99.83	99.35	99.54	99.17	98.10	99.94	98.99	99.30
	GRU	99.50	98.98	99.78	99.38	99.55	99.49	98.79	99.96	99.37	99.57
MI	CNN	99.87	99.74	99.94	99.84	99.89	98.38	99.42	96.54	97.63	98.08
	RNN	99.90	99.79	99.96	99.87	99.91	98.99	99.38	98.09	98.64	98.84
	LSTM	99.88	99.88	99.82	99.85	99.87	99.60	99.09	99.93	99.51	99.66
	GRU	99.91	99.82	99.96	99.89	99.92	98.47	99.45	96.73	97.79	98.18
Chi2	CNN	99.85	99.69	99.94	99.82	99.87	99.60	99.06	99.95	99.51	99.66
	RNN	99.88	99.77	99.94	99.86	99.89	98.81	99.22	97.81	98.39	98.64
	LSTM	99.87	99.75	99.93	99.84	99.88	99.60	99.16	99.85	99.50	99.64
	GRU	99.88	99.77	99.93	99.85	99.89	99.61	99.12	99.90	99.51	99.65
ANOVA	CNN	99.87	99.80	99.87	99.84	99.87	99.73	99.52	99.81	99.66	99.74
	RNN	99.84	99.70	99.89	99.80	99.85	97.54	99.72	94.12	95.78	96.97
	LSTM	99.85	99.79	99.84	99.82	99.85	95.78	99.61	89.86	89.73	94.80
	GRU	99.84	99.80	99.79	99.80	99.83	96.23	99.51	90.94	91.75	95.35
L1	CNN	99.54	98.87	100	99.43	99.62	98.97	98.99	98.43	98.65	98.88
	RNN	99.54	98.87	100	99.43	99.62	99.02	99.07	98.48	98.73	98.93
	LSTM	99.49	98.89	99.83	99.36	99.55	93.63	93.13	99.94	95.25	94.68
	GRU	99.50	98.89	99.87	99.38	99.57	99.26	98.38	99.84	99.09	99.35
Union	CNN	99.84	99.68	99.92	99.80	99.85	99.86	99.75	99.91	99.83	99.87
	RNN	99.87	99.73	99.94	99.84	99.88	99.84	99.72	99.89	99.81	99.85
	LSTM	99.87	99.75	99.93	99.84	99.88	98.55	99.59	96.80	97.91	98.26
	GRU	99.86	99.72	99.93	99.82	99.87	99.83	99.71	99.86	99.78	99.83
RFE	CNN	99.82	99.60	99.96	99.78	99.85	99.22	99.33	98.73	98.99	99.14
	RNN	99.85	99.75	99.87	99.81	99.85	98.65	99.21	97.41	98.16	98.44
	LSTM	99.79	99.49	100	99.74	99.83	98.81	98.03	99.06	98.53	98.85
	GRU	99.89	99.73	100	99.86	99.91	98.64	98.98	97.61	98.20	98.47

Table 10

Comparison of our proposed centralized learning approach with the results of [51].

Feature selection	model	Centralized - [51]					Centralized – Our Proposed				
		Accuracy	Precision	Recall	F1_Score	AUC_ROC	Accuracy	Precision	Recall	F1_Score	AUC_ROC
VT	CNN	96.24	96.51	99.58	98.02	73.50	99.52	98.90	99.89	99.40	99.58
	RNN	94.99	95.20	99.67	97.38	63.09	99.52	98.89	99.92	99.40	99.59
	LSTM	94.97	95.17	99.69	97.38	62.84	99.48	98.88	99.83	99.35	99.54
	GRU	94.84	94.92	99.83	97.31	60.89	99.50	98.98	99.78	99.38	99.55
MI	CNN	93.60	93.60	100	96.69	50.00	99.87	99.74	99.94	99.84	99.89
	RNN	97.74	97.99	99.63	98.80	84.87	99.90	99.79	99.96	99.87	99.91
	LSTM	97.45	97.50	99.84	98.66	81.21	99.88	99.88	99.82	99.85	99.87
	GRU	97.54	97.51	99.92	98.70	81.34	99.91	99.82	99.96	99.89	99.92
Chi2	CNN	93.84	94.19	99.56	96.80	54.90	99.85	99.69	99.94	99.82	99.87
	RNN	94.16	95.09	98.87	96.94	62.11	99.88	99.77	99.94	99.86	99.89
	LSTM	93.87	93.91	99.93	96.83	52.57	99.87	99.75	99.93	99.84	99.88
	GRU	94.23	95.19	98.83	96.97	62.90	99.88	99.77	99.93	99.85	99.89
ANOVA	CNN	92.80	95.08	97.34	96.20	61.85	99.87	99.80	99.87	99.84	99.87
	RNN	94.07	94.74	99.17	96.91	59.37	99.84	99.70	99.89	99.80	99.85
	LSTM	94.23	94.73	99.36	96.99	59.25	99.85	99.79	99.84	99.82	99.85
	GRU	94.13	94.72	99.26	96.94	59.18	99.84	99.80	99.79	99.80	99.83
L1	CNN	95.46	96.16	99.11	97.61	70.60	99.54	98.87	100	99.43	99.62
	RNN	95.05	95.17	99.78	97.42	62.86	99.54	98.87	100	99.43	99.62
	LSTM	95.09	95.33	99.63	97.43	64.14	99.49	98.89	99.83	99.36	99.55
	GRU	94.94	95.12	99.71	97.36	62.48	99.50	98.89	99.87	99.38	99.57
Union	CNN	99.84	99.68	99.92	99.80	99.85	99.84	99.68	99.92	99.80	99.85
	RNN	99.87	99.73	99.94	99.84	99.88	99.87	99.73	99.94	99.84	99.88
	LSTM	99.87	99.75	99.93	99.84	99.88	99.87	99.75	99.93	99.84	99.88
	GRU	99.86	99.72	99.93	99.82	99.87	99.86	99.72	99.93	99.82	99.87
RFE	CNN	97.05	96.99	99.95	98.45	77.35	99.82	99.60	99.96	99.78	99.85
	RNN	97.80	97.86	99.84	98.84	83.93	99.85	99.75	99.87	99.81	99.85
	LSTM	97.86	97.95	99.80	98.87	84.67	99.79	99.49	100	99.74	99.83
	GRU	97.87	97.90	99.87	98.87	84.25	99.89	99.73	100	99.86	99.91

and five clients. In Section 5.3, we conducted experiments with three clients, whereas in this section, we implemented the FL platform with five clients. The data distribution is provided in Table 7. Our findings indicate that the accuracy results generally decreased as the number of clients increased across all feature selection methods, following the implementation of four deep learning models, as shown in Table 12.

However, there were some exceptions. For instance, in the case of CNN using ANOVA, the accuracy with five clients (99.76 %) was slightly higher than with three clients (99.73 %). Similarly, LSTM using L1-based showed an accuracy of 93.65 % with five clients compared to 93.63 % with three clients, and GRU using L1-based achieved 99.52 % with five clients compared to 99.26 % with three clients.

Table 11

A comparison of our proposed federated learning approach with the state-of-the-art methods.

Authors	Year	Accuracy FL	Dataset	Classifier	No. of Clients
[47]	2021	70.40 %	NSL-KDD	Neural Networks with Greedy Feature Selection.	C= [8]
[56]	2021	78.46 %, 81.19 %	CICIDS2018, UNSW-NB15	CNN	C= [10,15]
[35]	2022	96.02 %, 96.19 %, and 96.39 %.	UNSW-NB15	ANN	C= [4,6,10]
[61]	2023	99.40 %	IoT-Botnet 2020	DNN with mutual information feature selection.	C= [8]
[50]	2023	93.00 %, 94.00 %	Edge-IIoTTest	CNN-RNN	C= [3, 9, 15]
Proposed Model	2025	99.73 %, 97.54 %, 95.78 %, and 96.23 %.	IoTID20	CNN-RNN-LSTM-GRU using ANOVA Feature Selection.	C= [3, 5]

Table 12

Accuracy of four DL models across various feature selection methods with three and five clients in the FL system.

Model	Number of clients	VT	MI	chi2	ANOVA	L1-based	Union	RFE
CNN	3clients	98.79	98.38	99.60	99.73	98.97	99.86	99.22
	5clients	96.97	95.69	96.24	99.76	98.13	96.43	95.50
RNN	3clients	98.72	98.99	98.81	97.54	99.02	99.84	98.65
	5clients	97.09	96.02	95.62	95.73	96.89	96.58	97.78
LSTM	3clients	99.17	99.60	99.60	95.78	93.63	98.55	98.81
	5clients	93.64	93.66	95.69	95.74	93.65	95.73	97.51
GRU	3clients	99.49	98.47	99.61	96.23	99.26	99.83	98.64
	5clients	99.40	96.71	98.49	93.81	99.52	95.77	98.02

5.6. Discussion

In this work, we proposed centralized and Federated Learning (FL)-based models for intrusion detection systems. Our findings suggest that FL achieved results like centralized learning across all feature selection techniques, with only a slight reduction in accuracy, which is outweighed by the significant privacy benefits FL offers. Additionally, the ensemble feature selection technique enhanced the model's ability to identify key features, leading to improved evaluation performance. Our experiments also demonstrated that the centralized learning approach outperformed other centralized IDS models for IoT datasets. However, the results indicated that as the number of clients in the FL platform increased, accuracy tended to decrease. This is likely due to the increased data diversity, suggesting that the aggregation methods may not effectively handle the highly heterogeneous data across clients, leading to a decline in FL system performance.

6. Conclusions and future work

In this study, we conducted training experiments to compare centralized and Federated Learning (FL) frameworks using various feature selection techniques across multiple deep learning models. Although the FL results were slightly lower than those of centralized learning, FL effectively preserved data privacy by processing the data locally instead of transmitting it to a central server. This approach not only ensured privacy but also reduced communication overhead, saved bandwidth, and sped up the training process. Both models were tested under identical conditions, including the same dataset and data size. When comparing our centralized model to others in the literature that used different model structures and parameter settings, our model outperformed others across all feature selection techniques and selected features.

In the centralized learning scenario, the highest accuracy across all experiments with different feature selection techniques was achieved using GRU with Mutual Information (MI) at 99.91 %, followed by RNN with MI at 99.90 %. Conversely, the lowest accuracy was observed with LSTM using Variance Threshold (VT) at 99.48 % and LSTM using the L1-based method at 99.49 %. In the FL scenario, the highest accuracy was achieved with CNN using the ANOVA method at 99.73 %, followed by GRU with Chi2 at 99.61 %. On the other hand, the lowest accuracy was recorded with LSTM using the L1-based method at 93.63 % and LSTM using ANOVA at 95.78 %.

For future work, we plan to explore adversarial attacks on one or more nodes in the FL framework and investigate methods to effectively filter poisoned models. Additionally, we suggest exploring advanced techniques to mitigate the impact of non-IID data across IoT devices, such as client selection strategies or adaptive aggregation methods.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRedit authorship contribution statement

Tasneem Qasem Al-Ghadi: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Selvakumar Manickam:** Writing – review & editing. **I. Dewa Made Widia:** Writing – review & editing. **Eka Ratri Noor Wulandari:** Writing – review & editing. **Shankar Karuppayah:** Writing – review & editing, Validation, Supervision, Project administration.

References

- [1] S. Agrawal, S. Sarkar, O. Aouedi, G. Yenduri, K. Piamrat, M. Alazab, S. Bhattacharya, P.K.R. Maddikunta, T.R. Gadekallu, Federated Learning for intrusion detection system: concepts, challenges and future directions, in: *Comput. Commun.*, Elsevier B.V., 2022, pp. 346–361, doi:[10.1016/j.comcom.2022.09.012](https://doi.org/10.1016/j.comcom.2022.09.012). Vol. 195.
- [2] L.A.C. Ahakonye, C.I. Nwakanma, J.M. Lee, D.S. Kim, SCADA intrusion detection scheme exploiting the fusion of modified decision tree and Chi-square feature selection, *IoT 21* (2023), doi:[10.1016/j.iot.2022.100676](https://doi.org/10.1016/j.iot.2022.100676).
- [3] Z. Ahmad, A.S. Khan, K. Nisar, I. Haider, R. Hassan, M.R. Haque, S. Tarmizi, J.J.P.C. Rodrigues, Anomaly detection using deep neural network for iot architecture, *Appl. Sci. (Switz.)* 11 (15) (2021), doi:[10.3390/app11157050](https://doi.org/10.3390/app11157050).
- [4] Z. Alebouyeh, A.J. Bidgoly, Privacy-preserving federated learning compatible with robust aggregators, *Eng. Appl. Artif. Intell.* 143 (2025), doi:[10.1016/j.engappai.2025.110078](https://doi.org/10.1016/j.engappai.2025.110078).
- [5] R. Anusuya, D.K. Renuka, FedAssess: analysis for efficient communication and security algorithms over various federated learning frameworks and mitigation of label-flipping attack, *Bull. Pol. Acad. Sci.: Tech. Sci.* 72 (3) (2024), doi:[10.24425/bpasts.2024.148944](https://doi.org/10.24425/bpasts.2024.148944).
- [6] M. Awad, S. Fraihat, Recursive feature elimination with cross-validation with decision tree: feature selection method for machine learning-based intrusion detection systems, *J. Sens. Actuator Netw.* 12 (5) (2023), doi:[10.3390/jsan12050067](https://doi.org/10.3390/jsan12050067).
- [7] B. P., Deepa, N., Pham, Q.-V., Nguyen, D.C., M. P.K.R., G. T.R., Pathirana, P.N., & Dobre, O. (2021). *Toward blockchain for edge-of-things: a new paradigm, opportunities, and future directions*. <https://doi.org/10.1109/IOTM.0001.2000191>

- [8] Benmalek, M., Ali Benrekia, M., Challal, Y., & Challal Security, Y. (n.d.). *Security of federated learning: attacks, defensive mechanisms, and challenges*. 2022(1), 49–59. <https://doi.org/10.18280/ria.360106i>
- [9] Briggs, C., Fan, Z., & Andras, P. (n.d.). *Federated learning with hierarchical clustering of local updates to improve training on non-IID data*.
- [10] Z. Chen, J. Li, L. Cheng, X. Liu, Federated-WDCGAN: a federated smart meter data sharing framework for privacy preservation, *Appl. Energy* 334 (2023), doi:10.1016/j.apenergy.2023.120711.
- [11] C.P. Chenet, A. Savino, S. Di Carlo, A survey on hardware-based malware detection approaches, *IEEE Access*. 12 (2024) 54115–54128, doi:10.1109/ACCESS.2024.3388716.
- [12] T.E.T. Djaidja, B. Brik, A. Boualouache, S.M. Senouci, Y. Ghamri-Doudane, Federated learning for 5G and beyond, a blessing and a curse- an experimental study on intrusion detection systems, *Comput. Secur.* 139 (2024), doi:10.1016/j.cose.2024.103707.
- [13] M.A. Elfaki, H.M. Alshahrani, K. Mahmood, R. Alabdan, M. Alymani, H. Alshahrani, A. Motwakel, A.A. Alneil, Metaheuristics algorithm-based minimization of communication costs in federated learning, *IEEE Access*. 11 (2023) 81310–81317, doi:10.1109/ACCESS.2023.3300221.
- [14] A.E.M. Eljaily, M.Y. Uddin, S. Ahmad, Novel framework for an intrusion detection system using multiple feature selection methods based on Deep learning, *Tsinghua Sci. Technol.* 29 (4) (2024) 948–958, doi:10.26599/TST.2023.9010032.
- [15] Z. Feng, Federated learning security threats and defense approaches, *Highlights Sci. Eng. Technol. CSIC* 2023 (2024).
- [16] M. Firat, M.G. Bakal, A. Akbaş, Machine learning based network intrusion detection with hybrid frequent item set mining, *J. Polytech.* (2023), doi:10.2339/po-liteknik.1386467.
- [17] O. Friha, M.A. Ferrag, M. Benbouzid, T. Berghout, B. Kantarci, K.K.R. Choo, 2DF-IDS: decentralized and differentially private federated learning-based intrusion detection system for industrial IoT, *Comput. Secur.* 127 (2023), doi:10.1016/j.cose.2023.103097.
- [18] G.S. Fuhnwi, M. Revelle, C. Izurieta, Improving network intrusion detection performance : an empirical evaluation using extreme gradient boosting (XGBoost) with recursive feature elimination, *2024 IEEE 3rd International Conference on AI in Cyber-security, ICAIC 2024*, 2024, doi:10.1109/ICAIC60265.2024.10433805.
- [19] S.K. Gebresilassie, J. Rafferty, L. Chen, Z. Cui, M. Abu-Tair, Transfer and CNN-based de-authentication (Disassociation) DoS attack detection in IoT wi-fi networks, *Electron. (Switz.)* 12 (17) (2023), doi:10.3390/electronics12173731.
- [20] E. Geo Francis, S. Sheeja, Intrusion detection system and mitigation of threats in IoT networks using AI techniques: a review, in: *Engineering and Applied Science Research*, Paulus Editora, 2023, pp. 633–645, doi:10.14456/easr.2023.66. Vol. 50, Issue 6.
- [21] V. Gugueoth, S. Safavat, S. Shetty, Security of Internet of Things (IoT) using federated learning and deep learning — Recent advancements, issues and prospects, in: *ICT Express*, Korean Institute of Communications and Information Sciences, 2023, pp. 941–960, doi:10.1016/j.icte.2023.03.006. Vol. 9, Issue 5.
- [22] M.J. Idrissi, H. Alami, A. El Mahdaoui, A. El Mekki, S. Oualil, Z. Yartaoui, I. Berrada, Fed-ANIDS: federated learning for anomaly-based network intrusion detection systems, *Expert. Syst. Appl.* 234 (2023), doi:10.1016/j.eswa.2023.121000.
- [23] T.N. Islam, S.R. Sabuj, Compensation layer-added federated learning receiver: design and implementation, *Internet Technol. Lett.* 5 (6) (2022), doi:10.1002/itl2.389.
- [24] A. Jahangeer, S.U. Bazai, S. Aslam, M. Marjan, M. Anas, S.H. Hashemi, A review on the security of IoT networks: from Network Layer's perspective, in: *IEEE Access*, Institute of Electrical and Electronics Engineers Inc, 2023, pp. 71073–71087, doi:10.1109/ACCESS.2023.3246180. Vol. 11.
- [25] O. Jullian, B. Otero, E. Rodriguez, N. Gutierrez, H. Antona, R. Canal, Deep-Learning based detection for cyber-attacks in IoT networks: a distributed attack detection framework, *J. Netw. Syst. Manag.* 31 (2) (2023), doi:10.1007/s10922-023-09722-7.
- [26] A. Kadhimi Ali, A. Mohsin Abdullah, S. Fawzi Raheem, Impact the classes' Number on the convolutional neural networks performance for image classification, *Int. J. Adv. Sci. Comput. Eng.* 5 (2) (2023).
- [27] S.M. Kasongo, A deep learning technique for intrusion detection system using a Recurrent Neural Networks based framework, *Comput. Commun.* 199 (2023) 113–125, doi:10.1016/j.comcom.2022.12.010.
- [28] M. Katayani, K. Keshamoni, A. Sree Rama Chandra Murthy, K. Usha Rani, S.L. Reddy, D. Kumar Alapati, Federated Learning: advancements, applications, and future directions for collaborative machine learning in distributed environments, *J. Electr. Syst.* 20 (5) (2024).
- [29] J. Li, X. Tong, J. Liu, L. Cheng, An efficient federated learning system for network intrusion detection, *IEEE Syst. J.* 17 (2) (2023) 2455–2464, doi:10.1109/JSYST.2023.3236995.
- [30] Q. Li, S. Zhao, S. Zhao, J. Wen, Logistic regression Matching pursuit algorithm for text classification, *Knowl. Based. Syst.* 277 (2023), doi:10.1016/j.knsys.2023.110761.
- [31] Q. Liu, Y. Huang, C. Jin, X. Zhou, Y. Mao, C. Catal, L. Cheng, Privacy and integrity protection for IoT multimodal data using machine learning and blockchain, *ACM Trans. Multimed. Comput. Commun. Appl.* 20 (6) (2024), doi:10.1145/3638769.
- [32] Z. Long, H. Yan, G. Shen, X. Zhang, H. He, L. Cheng, A Transformer-based network intrusion detection approach for cloud security, *J. Cloud Comput.* 13 (1) (2024), doi:10.1186/s13677-023-00574-9.
- [33] W.A. Mahdi, A.J. Obaidullah, Combination of machine learning and COSMO-RS thermodynamic model in predicting solubility parameters of cofomers in production of cocrystals for enhanced drug solubility, *Chemom. Intell. Lab. Syst.* 253 (2024), doi:10.1016/j.chemolab.2024.105219.
- [34] M. Mansour, A. Gamal, A.I. Ahmed, L.A. Said, A. Elbaz, N. Herencsar, A. Soltan, Internet of Things: a comprehensive overview on protocols, architectures, technologies, simulation tools, and future directions, *Energies*, MDPI, 2023 Vol. 16, Issue 8, doi:10.3390/en16083465.
- [35] W. Marfo, D.K. Tosh, S.V. Moore, Network anomaly detection using federated learning, in: *Proceedings - IEEE Military Communications Conference MILCOM, 2022-November*, 2022, pp. 484–489, doi:10.1109/MILCOM55135.2022.10017793.
- [36] Mazhar, T., Bux Talpur, D., Al Shoul, T., Ghadi, Y.Y., Haq, I., Ullah, I., Ouahada, K., & Hamam, H. (2023). *Analysis of IoT security challenges and its solutions using artificial intelligence*. <https://doi.org/10.3390/10.3390/brainsci13040683>
- [37] M. Mededjel, G. Belalem, F.Z.N. Benadda, S. Kadakelloucha, A blockchain application prototype for the internet of things, *J. Commun. Softw. Syst.* 18 (2) (2022) 124–136, doi:10.24138/JCOMSS-2021-0129.
- [38] I.D. Mienye, T.G. Swart, G. Obaido, Recurrent neural networks: a comprehensive review of architectures, variants, and applications, *Information* 15 (9) (2024) 517, doi:10.3390/info15090517.
- [39] B. Mohammed, E. Gbashi, Intrusion detection system for NSL-KDD dataset based on deep learning and recursive feature elimination, *Eng. Technol. J.* 39 (7) (2021) 1069–1079, doi:10.30684/etj.v39i7.1695.
- [40] A. Momand, S.U. Jan, N. Ramzan, A systematic and comprehensive survey of recent advances in intrusion detection systems using machine learning: deep learning, datasets, and attack taxonomy, *Journal of Sensors*, Hindawi Limited, 2023 Vol. 2023, doi:10.1155/2023/6048087.
- [41] S. Muhammad Salman Bukhari, M.H. Zafar, M.A. Houran, Z. Qadir, S. Kumayl Raza Moosavi, F. Sanfilippo, Enhancing cybersecurity in Edge IIoT networks: an asynchronous federated learning approach with a deep hybrid detection model, *Internet Things (Neth.)* 27 (2024), doi:10.1016/j.iot.2024.101252.
- [42] H.N.C. Neto, J. Hribar, I. Duspacic, D.M.F. Mattos, N.C. Fernandes, A survey on securing federated learning: analysis of applications, attacks, challenges, and trends, *IEE Access*. 11 (2023) 41928–41953, doi:10.1109/ACCESS.2023.3269980.
- [43] L. Ni, P. Huang, Y. Wei, M. Shu, J. Zhang, Federated learning model with adaptive differential privacy protection in medical IoT, *Wirel. Commun. Mob. Comput.* 2021 (2021), doi:10.1155/2021/8967819.
- [44] A. Nwosu, G. Imuetinyan Osaze Aimufua, B. Adeniyi Ajayi, M. Olalere, The impact of regularisation on linear regression based model, *J. Artif. Intell. Comput. Sci.* (2024), doi:10.5281/zenodo.13118703.
- [45] M.M. Obaid, M.H. Saleh, Efficient intrusion detection through the fusion of AI algorithms and feature selection methods, *J. Eng.* 30 (07) (2024) 184–201, doi:10.31026/j.eng.2024.07.11.
- [46] V. Pravylo, Y. Averkiev, Analysing malicious software supporting ddos attacks on iot networks, *Inf. Telecommun. Sci.* 1 (2024) 50–54, doi:10.20535/2411-2976.12024.50-54.
- [47] Y. Qin, M. Kondo, Federated learning-based network intrusion detection with a feature selection approach, *3rd International Conference on Electrical, Computer and Computer Engineering, ICECCE 2021*, 2021, doi:10.1109/ICECCE52056.2021.9514222.
- [48] F. Qiu, G. Zhang, X. Sheng, L. Jiang, L. Zhu, Q. Xiang, B. Jiang, P. Chen, Predicting students' performance in e-learning using learning process and behaviour data, *Sci. Rep.* 12 (1) (2022), doi:10.1038/s41598-021-03867-8.
- [49] S. Racherla, P. Sripathi, N. Faruqui, M. Alamgir Kabir, M. Whaiduzzaman, S. Aziz Shah, Deep-IDS: a real-time intrusion detector for IoT nodes using deep learning, *IEEE Access*. 12 (2024) 63584–63597, doi:10.1109/ACCESS.2024.3396461.
- [50] M.M. Rashid, S.U. Khan, F. Eusufzai, M.A. Redwan, S.R. Sabuj, M. Elsharief, A federated learning-based approach for improving intrusion detection in industrial Internet of Things networks, *NetWork* 3 (1) (2023) 158–179, doi:10.3390/network3010008.
- [51] S.D.A. Rihan, M. Anbar, B.A. Alabsi, Approach for detecting attacks on IoT networks based on ensemble feature selection and deep learning models, *Sensors* 23 (17) (2023), doi:10.3390/s23177342.
- [52] T. Saba, A. Rehman, T. Sadad, H. Kolivand, S.A. Bahaj, Anomaly-based intrusion detection system for IoT networks through deep learning model, *Comput. Electr. Eng.* 99 (2022), doi:10.1016/j.compeleceng.2022.107810.
- [53] S.R. Sabuj, M. Elsharief, H.S. Jo, A partial federated learning model in cognitive UAV-enabled edge computing networks, *Int. Conf. ICT Converg.* 2022-Oct. (2022) 1437–1440, doi:10.1109/ICTC55196.2022.9952980.
- [54] I.H. Sarker, A.I. Khan, Y.B. Abushark, F. Alsolami, Internet of Things (IoT) security intelligence: a comprehensive overview, machine learning solutions and research directions, *Mob. Netw. Appl.* 28 (1) (2023) 296–312, doi:10.1007/s11036-022-01937-3.
- [55] N.V. Sharma, N.S. Yadav, An optimal intrusion detection system using recursive feature elimination and ensemble of classifiers, *Microprocess. Microsyst.* 85 (2021), doi:10.1016/j.micpro.2021.104293.
- [56] J. Shi, B. Ge, Y. Liu, Y. Yan, S. Li, Data privacy security guaranteed network intrusion detection system based on federated learning, *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPS 2021*, 2021, doi:10.1109/INFOCOMWKSHPS51825.2021.9484545.
- [57] Souhila Guendouzi, B., Ouchani, S., El Assaad, H., El Zaher, M., Souhila GUENDOUZI, B., Assaad, H.E.L., & Zaher, M.E.L. (n.d.). A systematic review of federated learning: challenges, aggregation methods, and development tools A systematic review of federated learning: challenges, aggregation methods, and development tools A Systematic Review of Federated Learning: challenges, aggregation methods, and development tools. *J. Netw. Comput. Appl. (JNCA)*, 2023(9). <https://doi.org/10.1016/j.jnca.2023.103714i>
- [58] A. Thakkar, R. Lohiya, A review on machine learning and deep learning perspectives of IDS for IoT: recent updates, security issues, and challenges, *Arch. Comput. Methods Eng.* 28 (4) (2021) 3211–3243, doi:10.1007/s11831-020-09496-0.

- [59] D. Tollner, W. Ziyu, M. Zöldy, Á. Török, Demonstrating a new evaluation method on ReLU based Neural Networks for classification problems, *Expert. Syst. Appl.* (2024) 250, doi:[10.1016/j.eswa.2024.123905](https://doi.org/10.1016/j.eswa.2024.123905).
- [60] I. Ullah, Q.H. Mahmoud, A scheme for generating a dataset for anomalous activity detection in IoT networks, *Lect. Notes Comput. Sci. (Incl. Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinform.)* 12109 LNAI (2020) 508–520, doi:[10.1007/978-3-030-47358-7_52](https://doi.org/10.1007/978-3-030-47358-7_52).
- [61] X. Wang, Y. Wang, Z. Javaheri, L. Almutairi, N. Moghadamnejad, O.S. Younes, Federated deep learning for anomaly detection in the internet of things, *Comput. Electr. Eng.* 108 (2023), doi:[10.1016/j.compeleceng.2023.108651](https://doi.org/10.1016/j.compeleceng.2023.108651).
- [62] J. Wen, Z. Zhang, Y. Lan, Z. Cui, J. Cai, W. Zhang, A survey on federated learning: challenges and applications, *Int. J. Mach. Learn. Cybern.* 14 (2) (2023) 513–535, doi:[10.1007/s13042-022-01647-y](https://doi.org/10.1007/s13042-022-01647-y).
- [63] C. Xu, Y. Qu, Y. Xiang, L. Gao, Asynchronous federated learning on heterogeneous devices: a survey, *Computer Science Review, Elsevier Ireland Ltd*, 2023 Vol. 50, doi:[10.1016/j.cosrev.2023.100595](https://doi.org/10.1016/j.cosrev.2023.100595).
- [64] R. Xu, X. Liu, J. Wei, X. Ai, Z. Li, H. He, Predicting the deformation of a concrete dam using an integration of long short-term memory (LSTM) networks and Kolmogorov–Arnold networks (KANs) with a dual-stage attention mechanism, *Water (Switz.)* 16 (21) (2024), doi:[10.3390/w16213043](https://doi.org/10.3390/w16213043).
- [65] M. Yang, M.K. Lim, Y. Qu, X. Li, D. Ni, Deep neural networks with L1 and L2 regularization for high dimensional corporate credit risk prediction, *Expert. Syst. Appl.* (2023) 213, doi:[10.1016/j.eswa.2022.118873](https://doi.org/10.1016/j.eswa.2022.118873).
- [66] R. Yang, H. He, Y. Wang, Y. Qu, W. Zhang, Dependable federated learning for IoT intrusion detection against poisoning attacks, *Comput. Secur.* 132 (2023), doi:[10.1016/j.cose.2023.103381](https://doi.org/10.1016/j.cose.2023.103381).